

Universitatea Națională de Știință și Tehnologie
POLITEHNICA București
Facultatea de Automatică și Calculatoare, Departamentul
de Calculatoare



LUCRARE DE DIPLOMĂ

VisioScience3D

Coordonator Științific:

Prof.dr.ing Morar Anca
Andreea

Autor:

Dragomir Andrei-Mihai

București, 2025

National University of Science and Technology
POLITEHNICA Bucharest

Faculty of Automatic Control and Computers,
Computer Science and Engineering Department



BACHELOR THESIS

VisioScience3D

Scientific Adviser:

Prof.dr.ing Morar Anca Andreea

Author:

Dragomir Andrei-Mihai

Bucharest, 2025

Realizarea acestui proiect nu ar fi fost posibilă fără ajutorul și sprijinul Doamnei Prof.dr.ing Anca Andreea Morar. Mulțumesc pentru îndrumare, promptitudine și răbdare. Mulțumesc tuturor celorlalți profesori, laboranți și colegi care mi-au facilitat procesul de învățare și mi-au oferit suportul necesar de-a lungul acestor 4 ani, ajutându-mă în a-mi pune baze solide în domeniul ingineriei și calculatoarelor.

Sinopsis

Proiectul VisioScience3D vine ca un răspuns la nevoia de a crea un mediu de învățare interactiv și captivant pentru elevii din învățământul preuniversitar ce lipsește de pe piața curentă. Acesta îmbină tehnologii moderne pentru a crea un sistem de învățare bazat pe vizualizări 3D și simulări interactive, care să faciliteze înțelegerea conceptelor complexe din domeniul științelor exacte.

În momentul curent învățarea geometriei, fizicii, chimiei și altor discipline științifice se face prin metode tradiționale, care nu reușesc să capteze atenția elevilor. Prin acest proiect se dorește venirea în întâmpinarea acestei nevoi și oferirea unui mediu de învățare interactiv și captivant care să faciliteze activitatea didactică și să îmbunătățească rezultatele elevilor.

Proiectul VisioScience3D este o aplicație web care permite utilizatorilor să exploreze domeniul științelor exacte prin intermediul simulărilor interactive și vizualizărilor 3D. Acesta oferă o soluție inovatoare și complexă pentru elevi cât în egală măsură și pentru profesori, având posibilitatea să predea și să evalueze elevii prin intermediul acestuia rapid și eficient.

Abstract

The VisioScience3D project comes as a response to the need to create a learning environment that is interactive and engaging for pre-university students, which is currently missing from the market. It combines modern technology to build a learning system based on 3D visualizations and interactive simulations, designed to facilitate the understanding of complex concepts in the exact sciences.

At the moment, teaching geometry, physics, chemistry and other disciplines relies on traditional methods that fail to attract students attention. Through this project we aim to address this gap by offering an interactive and captivating learning environment that supports teaching activities and enhances student outcomes.

VisioScience3D is a web application that allows users to explore the STEM subjects through interactive simulations and 3D visualizations. It provides an innovative and comprehensive solution both for students and for teachers, enabling fast and efficient teaching and assessment.

Cuprins

Mulțumiri	i
Sinopsis	ii
Abstract	iii
1 Introducere	1
1.1 Context	1
1.1.1 Obiective	1
1.1.2 Susținere științifică	2
1.2 Soluția propusă	4
1.3 Rezultate obținute	5
1.4 Structura lucrării	6
2 Analiza cerințelor	7
2.1 Plaja de utilizatori	7
2.1.1 Categori	7
2.1.2 Profilul utilizatorului	7
2.2 Motivația proiectului	8
2.3 Dezvoltarea produsului	9
2.4 Cerințe funcționale	10
2.4.1 Cerințe funcționale pentru utilizatorii elevi	10
2.4.2 Cerințe funcționale pentru utilizatorii profesori	11
2.5 Cerințe non-funcționale	12
2.6 Limitări	13
3 Studiu de piață / Soluții existente	14
3.1 Alte soluții existente	14
3.1.1 Colorado Edu	14
3.1.2 Invatamate	15
3.1.3 Mozaweb	15
3.1.4 Iqboard	16
3.2 Raportarea la alte soluții	16
3.3 Motivația alegerii VisioScience3D	17

4	Tehnologii utilizate pentru front-end	18
4.1	Soluția de front-end	18
4.1.1	Framework principal	18
4.1.2	Framework 3D	20
4.1.3	Modelare 3D	21
4.2	Soluția UI/UX	22
4.2.1	Branding-ul proiectului	22
4.2.2	Experiența utilizatorului	24
4.3	Soluția de creare a scenelor 3D	25
4.3.1	Utilizarea tehnologiilor WebGL și Three.js	25
4.3.2	Integrarea cu frontend-ul	27
5	Tehnologii utilizate pentru back-end	28
5.1	Descrierea arhitecturii	28
5.1.1	Configurarea clusterului	28
5.1.2	Structurarea și crearea fișierelor YAML	29
5.1.3	Gestionarea clusterului	31
5.1.4	Rutarea și gestionarea traficului	32
5.2	Descrierea serviciilor	33
5.2.1	Descrierea API-urilor	34
5.3	Securitate	35
5.4	CI/CD	36
5.4.1	Port forwarding	36
5.4.2	Deployment	36
5.4.3	Metrici / Monitorizare	36
5.4.4	Avantajele folosirii Prometheus	36
5.4.5	Colectare Prometheus	37
5.4.6	Grafana Dashboard	37
5.5	Soluția de baze de date	39
5.5.1	Structura bazei de date	39
5.5.2	Securitatea datelor	41
5.5.3	Integrarea cu back-endul	41
6	Detalii de implementare	42
6.1	Back-end	42
6.1.1	Dezvoltare back-end	44
6.1.2	Implementarea serviciilor	44
6.1.3	Conectivitate	47
6.1.4	Procesarea codului în editorul interactiv	48
6.2	Front-end	48
6.2.1	Configurare front-end	49
6.2.2	Dezvoltare front-end	49
6.2.3	Implementare front-end	50
6.2.4	Conectivitate	51

7	Prezentarea funcționalităților finale ale aplicației	52
7.1	Înregistrare și autentificare utilizator	52
7.2	Explorarea meniului principal	52
7.3	Accesarea secțiunilor educaționale	54
7.4	Interacțiunea cu scenele 3D educaționale	55
7.5	Gestionarea profilului de profesor	56
7.5.1	Crearea de clase și gestionarea elevilor	56
7.5.2	Crearea de teste și gestionarea lor	56
7.5.3	Vizualizarea rezultatelor	56
7.6	Gestionarea contului de elev	56
7.6.1	Intrarea în clasele profesorului	56
7.6.2	Accesarea testelor și vizualizarea rezultatelor	56
7.6.3	Compilatorul interactiv de cod	57
7.6.4	Tabelul de elemente interactiv	58
7.6.5	Panoul profesorului	58
7.6.6	Recompensele elevilor	59
8	Evaluarea implementării	60
8.1	Introducere	60
8.2	Răspunsurile de posibili utilizatori	60
8.3	Evaluarea back-endului	61
8.3.1	Testarea back-endului	61
8.3.2	Monitorizarea back-endului	61
8.3.3	Evaluarea performanței back-endului	62
8.4	Evaluarea front-endului	62
8.4.1	Testarea front-endului	62
8.4.2	Monitorizarea front-endului	62
8.4.3	Evaluarea performanței front-endului	62
8.5	Testarea infrastructurii / platformei	63
8.6	Probleme întâmpinate	63
8.6.1	Backend & Kubernetes Ingress	63
8.6.2	Accesibilitate CI/CD Build & Rollout Workflow	63
8.6.3	Epuizarea memoriei GPU din cauza texturilor masive	63
8.6.4	Crearea fluxului de parsare a codului C++	64
8.6.5	Dimensiunea clusterului Kubernetes	64
8.6.6	Popularea și accesul extern la intrările în baza de date	64
8.7	Analiza metricilor	65
9	Concluzii și perspective	66
9.1	Concluzii	66
9.2	Dezvoltare viitoare	66
A	Anexe	68

Capitolul 1

Introducere

1.1 Context

Educația este un pilon și un domeniu de bază al societății, iar tehnologia joacă un rol din ce în ce mai important în acest sector. E nevoie de o adaptare a pieței din zona educațională oferită încă de la vârste relativ mici ale copiilor. Educația ar trebui să evolueze împreună cu tehnologia și copiii de astăzi să învețe să o folosească individual în scopuri educaționale și mai ales să lucreze împreună cu ea și nu împotriva. În special pe acest topic, într-o lume în care platformele sociale și mediul de interacțiune video sunt de bază pentru tineri, este esențial să se dezvolte soluții educaționale care să fie atractive și eficiente în procesul clasic de învățare.

1.1.1 Obiective

În acest context, problema pe care o abordăm este crearea unei platforme educaționale interactive care să integreze tehnologii moderne și simulări 3D, pentru a face din procesul de învățare o experiență captivantă și eficientă pentru elevi de gimnaziu și liceu. Această platformă va permite accesul ușor și rapid la informații complexe de matematică, fizică, chimie, astronomie și informatică.

Studentii vor putea explora concepte științifice și interacționa cu simulări 3D, dar și să participe la teste și evaluări pentru a-și verifica cunoștințele. De asemenea, profesorii vor avea la dispoziție un instrument simplu pentru a crea testele și a gestiona clasele pe care le creează, facilitând procesul de predare și evaluare. Platforma va avea la final un sistem interactiv de navigare, recunoaștere a rezultatelor și răsplătire a progresului utilizatorilor prin gamificare tot folosind interacțiuni cu scene 3D, fapt ce va îmbunătăți

experiența utilizatorilor și va stimula învățarea activă și dinamică.

Obiectivele principale ale acestui proiect sunt:

- Crearea unei platforme educaționale interactive care să integreze simulări 3D.
- Folosirea unor tehnologii moderne și ușor de menținut pentru viitorului platformei.
- Dezvoltarea unui sistem de gestionare a testelor și evaluărilor ușor de folosit pentru profesori și elevi.
- Implementarea unui sistem de gamificare pentru a stimula învățarea activă și implicarea elevilor în învățare.
- Asigurarea accesibilității și ușurinței în utilizare pentru elevi și profesori.
- Crearea unui mediu de învățare captivant și eficient care să faciliteze înțelegerea conceptelor complexe.
- Integrarea unui sistem de raportare și monitorizare a progresului elevilor.
- Crearea unei interfețe prietenoase și intuitive care să faciliteze experiența profesorilor în gestionarea claselor, testelor, elevilor și a rezultatelor.

Proiectul determină evoluții importante în stilurile de predare și învățare în România, acoperind o nevoie nesatisfăcută încă de vreo platformă locală, gratuită și cu potențial bun de creștere și extindere.

1.1.2 Susținere științifică

Multe discipline STEM (matematică, fizică, chimie, etc) implică concepte abstracte foarte dificil de vizualizat, ceea ce poate scădea interesul elevilor. De exemplu, chimia este adesea percepută ca foarte abstractă deoarece elevii nu pot vizualiza conceptele precum structura moleculară sau reacțiile chimice, ele rămânând la nivel de simboluri într-o ecuație pe o foaie care nu spun prea multe despre realitatea și importanța conceptelor în domeniile științifice și ingineresti avansate.

Integrarea vizualizărilor 3D și a tehnologiilor interactive în predarea disciplinelor STEM este susținută de un număr mare de cercetări recente. Majoritatea au ajuns la concluzia că reprezentările vizuale și simulările contribuie la înțelegerea conceptelor abstracte, sporind și motivația elevilor.

De exemplu, un studiu derulat în școlile din Cehia a arătat că utilizarea modelelor

3D și animațiilor în predarea materiilor științifice a dus la o creștere în implicarea elevilor și în performanța la teste, în special în chimie și biologie [1]. De asemenea, o meta-analiză recentă a concluzionat că lecțiile care includ modele 3D interactive au îmbunătățit de peste 1,6 ori varianta standard de învățare teoretică [4]. Deci vorbim de o creștere de 60% raportată a notelor elevilor și totodată a satisfacției în învățare a celor din grupul care a folosit vizualizări 3D și a lucrat pe bază de probleme interactive. Așadar înțelegerea și învățarea conceptelor a fost mai ușoară dar și mai plăcută în cazul grupului supus la astfel de tipuri de a studia, ceea ce justifică existența și dezvoltarea de platforme care susțin și înaintează progresul în domeniu.

Alte publicații arată ca simulările 3D aplicate și în zona de laboratoare interactive au condus nu doar la o înțelegere mai bună a subiectelor, ci și la o retenție îmbunătățită a cunoștințelor de-a lungul timpului [5]. Elevii au raportat un nivel mai ridicat de încredere în propriile abilități și o atitudine mai pozitivă față de învățare, ceea ce e cel mai important aspect pentru a crea o generație care știe să se adapteze și să aibă încredere în propriile capacități.

Chiar și mai mult, numeroase alte cercetări evidențiază importanța predării adaptate stilurilor specifice de învățare ale fiecărui elev. Datele arată că un procent semnificativ dintre elevi învață predominant în mod vizual, ceea ce justifică utilizarea elementelor grafice și a animațiilor în predarea la clasă [2], [3]. Un studiu local desfășurat chiar în România confirmă această tendință, indicând o pondere de aproximativ 48% pentru stilul vizual de învățare dintre toate incluse în analiză, ceea ce subliniază necesitatea diversificării suportului educațional și includerea preponderent a elementelor vizuale în predare [2].

Există și un raport OCDE¹ care a demonstrat că utilizarea controlată a tehnologiei digitale în procesele educaționale poate conduce la o creștere cu până la 15% a scorurilor obținute de elevi la testele de competențe, comparativ cu metodele clasice [7].

Un alt studiu susține și el impactul pozitiv al învățării vizuale asupra performanței elevilor. Arată o creștere rapidă a utilizării VR/AR în învățarea STEM la elevi, cu efecte pozitive asupra implicării și performanței [8].

De asemenea, și alte cercetări au demonstrat că utilizarea învățării vizuale îmbunătățește eficiența educațională și crește motivația elevilor. În cadrul unui studiu, s-a demonstrat că elevii care învață prin intermediul reprezentărilor vizuale au o înțelegere mai clară a gândirii spațiale, relațiilor conceptuale și recunoașterea de modele [9].

¹Organizația pentru Cooperare și Dezvoltare Economică

Un alt studiu confirmă faptul că utilizarea vizualizărilor 3D în educație contribuie la gestionarea mai facilă a elevilor la clasă și la construirea unor bibliografii de resurse mai dezvoltate. Asta se poate observa și în articolul [11] care prezintă 5 avantaje ale platformelor educaționale online.

Pe partea rolului de profesor, există studii care arată că satisfacția profesorilor cu privire la utilizarea platformelor educaționale este strâns legată de interacțiunile eficiente cu elevii și de încrederea în propriile abilități tehnice [10]. O platformă ușor de utilizat poate contribui la o predare mai eficientă și mai motivantă, poate spori implicarea elevilor și poate facilita evaluarea continuă a progresului acestora. Astfel profesorii pot câștiga încredere să o folosească după ce vad progresul elevilor după cum arată și studiul [10] și să raporteze și ei satisfacție crescută în procesul de predare.

Ca și concluzie, publicațiile prezentate sugerează și susțin că integrarea vizualizărilor 3D și a simulărilor interactive nu doar crește nivelul de atractivitate a învățării, ci și eficiența ei la nivel individual al elevilor. Proiectul *VisioScience3D* se aliniază cu aceste direcții moderne de predare ca o soluție locală zonei României și care răspunde nevoilor curente din educație ale noii generații de elevi, oferind resurse educaționale care comunică pe limba lor și se axează pe retenție și modelarea procesului de învățare pe termen lung.

1.2 Soluția propusă

Ideea platformei este de a crea un mediu de învățare interactiv care să integreze simulări 3D și tehnologii moderne pentru a face procesul de învățare mult mai ușor. Oferă o gamă de materii care pot fi studiate prin această metodă, dar poate funcționa și ca verficator ad-hoc al cunoștințelor elevilor după predare sau la clasă. Un elev poate intra rapid și facil să verifice o formulă sau altă informație, iar profesorul poate să creeze teste și să gestioneze clasele de elevi în mod rapid și eficient.

Numele VisioScience3D a fost ales pentru a reflecta scopul platformei și este compus din două cuvinte: "Visio" care se referă în mod trivial la vizual, vedere, iar "Science" care se traduce direct în știință. Această combinație sugerează o platformă care îmbină ideea de vizual cu știința, oferind un nume care reflectă esența platformei și scopul său. Titlul conține și termenul 3D, care subliniază focusul vizualizărilor din platformă, cel de a le crea aproape exclusiv în spațiu tridimensional.

Fiecare rol din procesul educațional (elev, profesor) are posibilitatea de a accesa funcțiile de învățare și evaluare. De asemenea, platforma va avea un sistem de gamificare care

va recompensa elevii pentru progresul lor și va încuraja participarea activă. Opțiuni de vizualizare de tabele și grafice cu rezultatele vor fi disponibile pentru profesori. Elevii vor putea să-și urmărească scorul și progresul în timp real la teste și evaluări și se pot întoarce la conținutul educațional pentru a-și revizui cunoștințele dacă e nevoie.

Testele pot fi create prin selectare și mutare de elemente create ca întrebări, răspunsuri, puncte și altele în interfața dedicată secțiunii de creare a testelor, unde există control granular de la nivel de structură a quizului până la nivel de întrebare, răspunsuri corecte sau punctaje. Profesorii pot vizualiza clasele pe care le dețin, elevii implicați și rezultatele obținute de aceștia. De asemenea, profesorii pot vizualiza și analiza rezultatele elevilor pentru a înțelege mai bine progresul acestora și pentru a adapta metodele de predare în funcție de nevoile fiecărui elev. Această analiză va permite o abordare personalizată a învățării. Astfel acesta este de fapt scopul principal de evaluare care se propune în platformă. Profesorii sunt și singurii care au acces și la sistemul de invitații al elevilor în clase care funcționează direct către contul elevului.

Elevii pot accesa platforma printr-o interfață axată pe interactivitate și intuitivitate, unde simulările 3D și animațiile interactive. Din meniul 3D principal se selectează materia la care pot interacționa cu toată gama de simulări disponibile. De asemenea, elevii pot participa la teste și evaluări pentru a-și verifica cunoștințele. Aceste teste sunt concepute pentru flexibilitate, fiind agnostice de domeniu și potrivite pentru o testare rapidă după o lecție predată de exemplu.

1.3 Rezultate obținute

Platforma a ajuns într-un punct în care poate fi utilizată de către profesori și elevi pentru a explora concepte și reprezintă o soluție care poate salva timp și poate face învățarea mai rapidă și intuitivă pentru elevii cu stil de învățare vizual (preponderent cu peste 48% incidență în România). Pentru profesori platforma reprezintă curent o soluție rapidă de testare și monitorizare a elevilor la materii clasice cum ar fi fizica, chimia, matematica, dar și informatica sau astronomia.

La nivel tehnic, platforma folosește o arhitectură cu microservicii în back-end, oferind o scalabilitate foarte ridicată și o disponibilitate crescută datorită separării dintre servere și gestionarea traficului separat.

Arhitectura poate fi ușor extinsă pentru a adăuga noi funcționalități și module, iar platforma poate fi adaptată rapid la nevoile utilizatorilor. De asemenea, partea de front-end este construită folosind tehnologii cu suport extins și comunități mari, ceea

ce asigură o posibilă dezvoltare durabilă a platformei în viitor.

1.4 Structura lucrării

Această lucrare este structurată în mai multe capitole, fiecare abordând un aspect diferit al proiectului atât din punct de vedere tehnic, cât și din punct de vedere al implementării și utilizării platformei.

Capitolul 2 oferă o analiză detaliată a cerințelor și a nevoilor utilizatorilor, precum și a profilului utilizatorilor tipici ai platformei. Acesta include o descriere a motivației, a cerințelor funcționale și a celor non-funcționale, precum și a limitărilor și constrângerilor proiectului.

Al treilea capitol (3) analizează piața și competiția, evidențiind punctele forte și slabe ale acestora și modul în care platforma propusă se diferențiază de cea dezvoltată în cadrul acestui proiect.

Capitolul 4 detaliază tehnologiile utilizate în dezvoltarea platformei, inclusiv limbajele de programare, framework-urile și instrumentele. Aici se oferă o privire de ansamblu asupra dezvoltării fiecărei părți importante a aplicației: back-end, front-end (inclusiv UI/UX), scene 3D și bazele de date.

Capitolul 6 oferă detalii despre implementarea platformei, inclusiv bucăți de cod. Aici se include și detalii despre configurarea și conectivitatea obținută între diferitele componente ale aplicației.

Cel de-al șaptelea capitol (7) are ca focus definirea și prezentarea funcționalităților principale ale platformei, inclusiv modul în care utilizatorii pot interacționa cu aplicația și cum pot crea conturi, teste, clase și cum pot vizualiza scene și experimente 3D.

Capitolul 8 se concentrează pe tipurile de evaluare din platformă, cu accent pe testare și pe modul în care se poate monitoriza sănătatea serviciilor. De asemenea se discută și despre evaluare performanței platformei pe diferite niveluri.

Ultimul capitol, 9, oferă concluzii și perspective asupra viitorului platformei, inclusiv posibile îmbunătățiri și extinderi ale funcționalităților existente. De asemenea, se discută despre impactul pe care platforma ar putea să-l aibă în educația preuniversitară.

Capitolul 2

Analiza cerințelor

2.1 Plaja de utilizatori

Publicul țintă al aplicației este destul de general, putând fi accesată de orice utilizator care se înregistrează și dorește să învețe sau să își amintească noțiuni de matematică, fizică, chimie, astronomie sau informatică.

2.1.1 Categori

Categoriile principale de utilizatori suportate de platformă sunt utilizatorii elevi și profesorii, fiecare având un rol specific și având acces la funcționalități diferite.

Aceste categorii indică și publicul țintă al aplicației, care este format din elevi de nivel gimnaziu sau liceu și profesori care predau disciplinele menționate la acest nivel de învățământ.

Între aceste două categorii de utilizatori diferă drepturile de acces, tipul de interacțiune cu aplicația dar și funcționalitățile disponibile.

2.1.2 Profilul utilizatorului

După cum am menționat anterior, aplicația este destinată elevilor și profesorilor de matematică, fizică, chimie, astronomie și informatică. Putem face și o analiză din diverse puncte de vedere demografice și comportamentale:

Vârstă: Aplicația este destinată elevilor de gimnaziu și liceu (10-18 ani) și profesorilor de toate vârstele (25-60 ani).

Tip de învățare: Preponderent vizual. Se folosesc animații 3D pentru a explica conceptele științifice.

Studii: Utilizatorii sunt elevi de gimnaziu și liceu, respectiv profesori cu studii superioare în domeniul educației sau al științelor exacte. Scopul aplicației și publicul țintă explică și nivelul educațional al acestora.

Locație: Aplicația este destinată utilizării în România, iar limba folosită este limba română.

Interese: Utilizatorii au interese în domeniile educației, tehnologiei, științei și învățării interactive, domenii pe care aplicația le acoperă.

Motivație elevi: Elevii doresc să învețe și să își îmbunătățească cunoștințele în domeniile științifice sau să obțină performanțe mai bune la școală și liceu.

Motivație profesori: Profesorii doresc să își îmbunătățească metodele de predare, oferind elevilor o experiență mai interactivă și captivantă.

Tehnologie: Utilizatorii trebuie să aibă cel puțin un nivel de competență tehnologică începător-mediu și să fie familiarizați cu utilizarea platformelor web.

Dispozitive: Aplicația poate fi accesată de pe computere, laptopuri, tablete sau telefoane mobile.

2.2 Motivația proiectului

Motivația proiectului este de a oferi o platformă educațională interactivă care să ajute elevii, decizia ideii fiind luată după ce s-a studiat piața de soluții existente care oferă acest tip de produs destinat României.

Pentru a justifica mai mult nevoia de o asemenea platformă deschisă publicului de elevi și profesori din România am realizat un studiu scurt printr-un chestionar care a avut un număr de peste 35 de răspunsuri. Chestionarul a fost trimis către elevi și studenți, cât și către adulți care au răspuns la un nivel ipotetic dacă s-ar mai afla în școală sau liceu. Media de vârstă a respondenților a între 18 și 24 de ani, dar există și răspunsuri de la persoane de peste 35 de ani sau sub 14 ani.

Răspunsurile au indicat că peste 85% au ca stil de învățare principal stilul vizual, urmat de stilul auditiv și apoi prin citit/scriș, de menționat fiind că a fost o încrebarea cu răspuns multiplu. Asta susține motivația dezvoltării unei astfel de platforme ce are ca scop principal învățarea prin simulări 3D.

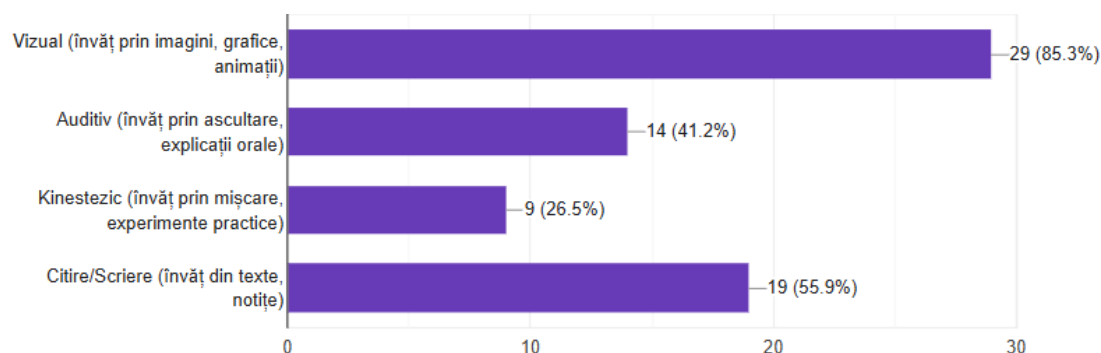


Figura 2.1: Stilurile de învățare preferate de respondenți

La întrebarea care materii ar fi util să existe într-o platformă de simulări didactice 3D, peste 75% dintre respondenți au spus fizică, peste 65% chimie, peste 55% matematică și la fel și pentru informatică, iar pentru biologie peste 60% dintre răspunsuri au menționat-o.

2.3 Dezvoltarea produsului

Dezvoltarea produsului s-a realizat în manieră iterativă, bazat pe testarea progresivă a funcționalităților implementate și feedback-ul primit de la utilizatorii pe care i-am avut la dispoziție în timpul dezvoltării. De fiecare dată s-au reevaluat cerințele și reprioritizat implementarea lor până la realizarea unui MVP ¹ satisfăcător care să poată fi menținut și dezvoltat cu alte funcționalități ulterior.

Cerințele și dorințele clasei țintă de utilizatori au fost colectate prin discuții directe și cu elevii din clasele unde este profesor unul dintre părinții autorului. Elevii au prezentat un entuziasm crescut pentru existența unei astfel de platforme gratuite și au dat sugestii după folosirea prototipului. Printre feedback-ul principal primit la momentul respectiv s-au numărat:

- Animațiile 3D sunt atractive și ușor de utilizat.
- Ar fi utilă o interacțiune mai consistentă cu scenele.
- Testele sunt eficiente, iar rezultatele sunt clare și ușor de vizualizat.
- O idee interesantă ar fi un sistem de recompense sub formă de obiecte sau insigne 3D pentru realizările utilizatorilor.
- Pe partea de algoritmică, un motor care să vizualizeze cum se modifică structurile de date în timp real ar adăuga un plus de valoare.

¹Produs cu funcționalitate minim valabilă

Principii de bază ale interfeței:

Interfața a fost realizată având la bază trei principii fundamentale. În primul rând, simplitatea a fost esențială, alegându-se un design minimalist, pentru a nu distrage atenția de la conținutul educațional. Al doilea principiu este interactivitatea dată de animațiile 3D din cât mai multe zone ale aplicației. Alt aspect important a fost accesibilitatea, folosindu-se o paletă de culori care să fie ușor de privit și care să nu obosească ochii utilizatorului.

2.4 Cerințe funcționale

Cerințele funcționale ale aplicației:

Cerințele funcționale ale aplicației sunt împărțite în două mari categorii, în funcție de tipul de utilizator.

2.4.1 Cerințe funcționale pentru utilizatorii elevi

Cerințele funcționale pentru utilizatorii elevi sunt cerințe care țin de modul în care utilizatorii pot să interacționeze cu aplicația din poziția de elevi.

Printre cele mai importante funcționalități se numără:

- Înregistrare și autentificare (crearea rapidă a unui cont propriu și acces securizat ulterior prin parolă, securizarea datelor personale prin criptare în bazele de date)
- Navigare în meniul 3D principal (alegerea materiei dintr-un meniu virtual care îndeamnă la explorarea tuturor secțiunilor disponibile ca într-un joc interactiv)
- Acces la module educaționale (vizualizarea modulelor structurate prin vizualizări 3D, accesul la toate informațiile educaționale încărcate în platformă, inclusiv texte, imagini și animații)
- Interacțiune cu scenele 3D (manipulare și observare în timp real a efectelor asupra modelelor, schimbare orientării, zoom, modelarea pieselor și alte acțiuni interactive)
- Aderare la clase prin invitație (acceptarea invitațiilor direct din profil pentru acces la clasele definite de profesori și vizualizarea testelor și structurii claselor în care se află)

- Gestionare invitații (notificări pentru invitații noi și posibilitatea de a accepta sau refuza direct din panoul profilului)
- Vizualizare și rezolvare teste (lista testelor active, vizualizarea rezultatelor și acces la rezolvarea testelor deschise, chiar și acces la retestare în caz că se permite acest lucru)
- Consultare rezultate (panouri cu scoruri proprii și ale colegilor, dacă sunt publice, acces la zona de recompense)
- Administrare profil (gestionarea autentificării și a datelor personale, accesul din profil la clase și la zona de recompense)

2.4.2 Cerințe funcționale pentru utilizatorii profesori

Acestea sunt cerințe care sunt necesare pentru a asigura o experiență adecvată utilizatorilor din rolul de profesor.

Printre funcționalitățile esențiale pentru profesori se numără:

- Înregistrare cont (crearea rapidă a unui profil de profesor cu acces elevat la toate funcționalitățile platformei)
- Autentificare securizată (acces pe bază de email și parolă, stocarea sigură a datelor personale în bazele de date și a parolei criptate)
- Meniu 3D principal (alegerea materiei din interfața virtuală, posibilitatea de a explora toate secțiunile disponibile ca și rolul de elev)
- Acces la resurse educaționale (consultarea materialelor și a modulelor din toate zonele și disciplinele disponibile)
- Scene 3D interactive (explorare și manipulare de vizualizări tridimensionale ca și rolul de elev)
- Gestionare elevi și clase (creare, editare și organizare de grupuri de elevi denumite „clase”, la care pot adera elevii prin invitație)
- Gestionare teste (configurare structură și parametri de evaluare, crearea testelor cu răspuns simplu sau răspuns multiplu)
- Deschidere/închidere teste (controlul perioadelor de acces la evaluări, permițând elevilor să rezolve testele doar când sunt deschise)

- Vizualizare rezultate (panouri dedicate cu scorurile elevilor și cu multe alte metrice de performanță, inclusiv statistici pe clase sau pe teste)
- Analiză și administrare rezultate (filtrare, sortare și analiză a rezultatelor elevilor)
- Meniu invitații (listarea și urmărirea statusului invitațiilor trimise către elevi)
- Trimitere invitații (distribuirea directă către elevi din profil, ei primind notificarea tot direct în profil)
- Administrare profil profesor (actualizarea datelor și conexiunii la platformă, accesul din profil la panourile cu clasele și testele create și la cele cu rezultate și performanțe)

2.5 Cerințe non-funcționale

Dezvoltarea produsului a fost destinată pentru utilizare pe Web, de pe orice dispozitiv cu acces la internet, fără a necesita instalarea de aplicații sau plugin-uri suplimentare. Din natura aplicației, gradul de portabilitate de la dispozitiv la dispozitiv este ridicat.

Invitațiile trebuie să ajungă în timp real (livrare aproape instantanee către destinatari). Încărcarea scenelor 3D trebuie să se facă în sub câteva secunde, iar latența să fie redusă în UI (răspuns rapid la orice acțiune a utilizatorului).

Răspunsul trebuie să fie instant la scenele 3D (aplicarea imediată a comenzilor de interacțiune), iar rezultatele să fie primite pe loc (afișarea scorului imediat după trimiterea testului) și gestionarea utilizatorilor să se facă prin autentificare sigură și administrare eficientă a datelor. Controlul stării testelor (deschidere sau blocare) trebuie să aibă efect imediat.

Pentru o utilizare mai intuitivă sunt adăugate în unele locuri și indicatori vizuali care arată modul de funcționare. Există numeroase exemple unde utilizatorul este îndrumat să pună mouse-ul peste un obiect pentru a activa meniuri sau este ghidat prin interfață.

Meniurile conțin informații de prezentare și tutoriale pentru a naviga diferitele simulări 3D, ele fiind primul lucru pe care utilizatorul le vede la intrarea în diferitele secțiuni. Datele afișate în platformă sunt și ele actualizate constant și dinamic prin comunicare cu serverul care gestionează acel serviciu în zona de back-end, prin protocolul HTTP.

Ca cerințe non-funcționale necesare pentru încărcarea și gestionarea corectă a scenelor 3D, se poate discuta pe mai multe planuri.

Browser & API

Aplicația este destinată browserelor moderne care suportă WebGL 1.0 (OpenGL 2.0) sau, de preferat, WebGL 2.0 (OpenGL 3.0). Browsers compatibile: Chrome > 60, Firefox > 52, Safari > 11.

CPU

Recomandat minim un procesor dual-core, > 1,5 GHz, majoritatea procesoarelor moderne fiind capabile să ruleze platforma fără probleme.

Memorie (RAM)

Recomandat minim 2 GB RAM total, din care 256 MB cel puțin liberi pentru heapul folosit de JavaScript și Three.js pentru încărcat modele și texturi.

GPU & VRAM

Minim placă grafică¹:

AMD/ATI: > Radeon 4xxx, NVIDIA: > GeForce 2xx, > Quadro FX 3800, Intel: > HD 4000

Rețea

Recomandat lățime de bandă de 1 Mbps pentru a încărca rapid obiectele și texturile.

Performanță în aplicație

FPS țintă: peste 30 fps, preferabil peste 60 fps pe desktop. Timp de încărcare inițială: sub 2s și sub 5 s (la texturi mari).

2.6 Limitări

Din punct de vedere al specificului platformei, ea oferă câteva limitări. Compatibilitate browser-platformă (aplicația este compatibilă cu majoritatea browserelor de astăzi, însă nu este optimizată pentru browserele mai vechi), performanță hardware (aplicația necesită un hardware minim pentru a funcționa corect), conexiune la internet (aplicația necesită o conexiune la internet stabilă pentru a funcționa corect), dispozitive mobile și tabletă (aplicația necesită un hardware decent pentru rulare, ceea ce este mai greu de obținut pentru dispozitivele mobile), practice (aplicația are limitări în utilizare date de capacitățile utilizatorului care o folosește), conținut (aplicația nu oferă un conținut educațional complet, ci doar partea modelabilă 3D a acestuia pe fiecare materie).

¹<https://enterprise.arcgis.com/en/system-requirements/10.5/windows/scene-viewer-requirements.htm>

Capitolul 3

Studiu de piață / Soluții existente

3.1 Alte soluții existente

3.1.1 Colorado Edu

Prima soluție analizată este Colorado Edu, o platformă care oferă multiple materii.

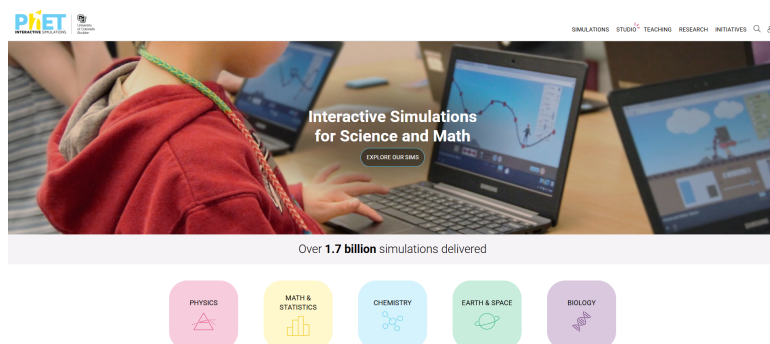


Figura 3.1: Captură de ecran de pe soluția Phet Colorado Edu

Se poate observa designul simplu și intuitiv și suportul relativ extins. Are și o interactivitate bună la nivel de simulări și alt avantaj e că are acces gratuit. Totuși, nu are un sistem de evaluare integrat și nu permite crearea de teste personalizate. De asemenea, nu are un sistem de management al utilizatorilor, ceea ce face ca utilizarea platformei să fie limitată la simulări ad-hoc, iar ținta principală de curriculum este SUA și nu România.

3.1.2 Invatamate

Cea de-a doua soluție analizată este invatamate.com, o platformă care oferă o suită de cursuri, vizualizări și simulări online. Platforma are un design relativ depășit și nu foarte atractiv, dar conține multiple resurse utile. Deși numele sugerează că platforma este dedicată matematicii, ea are și simulări de astronomie și joculețe interactive.

Problema este iar că nu are un sistem de gestionare al utilizatorilor și nici suport pentru evaluare integrat. Lipsa acestor funcționalități face ca platforma să nu fie foarte utilă în mediul educațional, iar utilizarea ei să fie limitată la joculețe simple și triviale. Alte limitări ale platformei sunt că folosește extensia Flash pentru majoritate jocurilor și chiar integrări cu Kahoot pentru unele, iar resursele sunt de asemenea legate de surse externe în unele cazuri.

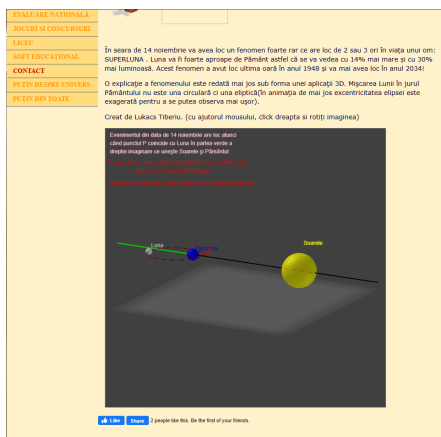


Figura 3.2: Simulări de pe platforma invatamate



Figura 3.3: Jocuri de pe platforma invatamate

3.1.3 Mozaweb

Această soluție are o arhitectură cu produse pentru mai multe roluri (pentru elevi, pentru profesori, pentru școli) și o experiență de utilizare plăcută și intuitivă.

Are suport pentru numeroase materii și din zona STEM, dar și pentru alte domenii. De asemenea, platforma are simulări foarte profesionale și interactive, dar dezavantajul mare e dat de prețurile abonamentelor pentru că discutăm de o platformă comercială și nu una gratuită. De asemenea pentru simulări este necesar să se instaleze un plugin extern, reducând astfel accesibilitatea platformei.

Ținta acestei platforme este una comercială mai mult decât educațională, iar prețurile sunt relativ mari pentru toate tipurile de utilizatori.

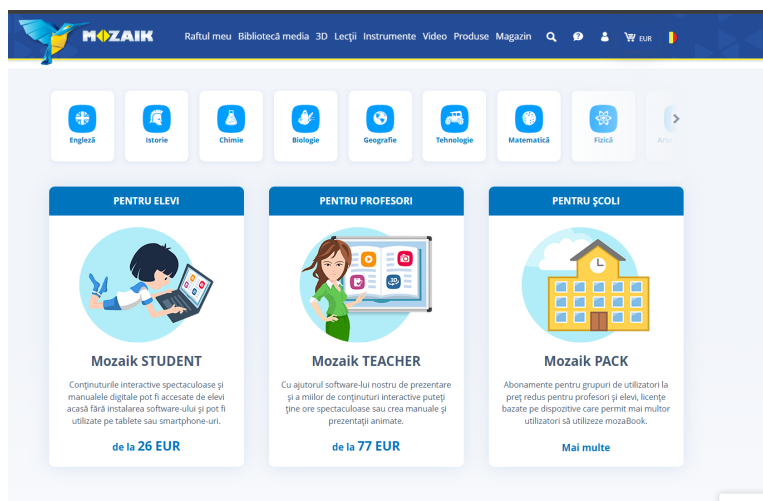


Figura 3.4: Captură de ecran de pe platforma mozaweb

3.1.4 Iqboard

Ultima soluție analizată este iqboard.ro, o platformă care oferă o suită de resurse educaționale în limba română, cu multiple moduri de lucru bazată pe un sistem de abonamente. Are o interfață interesantă și atractivă și suport extins educațional pe mai multe domenii.

Dezavantajele majore sunt prețurile extrem de mari pentru utilizatori (la nivel de sute de euro fiecare plan). Materialele sunt de calitate bună și platforma are un design interactiv și atractiv, dar prețul e un punct important. În figura 3.5 se poate urmări un exemplu de simulare disponibilă. Platforma are moduri separate pentru pregătire, predare și mod desktop, dar nu are un sistem de management al utilizatorilor sau vreun sistem de evaluare sau urmărire a progresului utilizatorilor.

Ca alte funcționalități, platforma se laudă cu modul multi-user care poate fi folosit în modul tablă sau full-screen. După selectarea instrumentului dorit, utilizatorii pot lucra simultan pe tablă, cu modul multi-screens (modul petrecere), cu posibilitate de folosire simultană a doua table interactive (modul MX2), dar și cu modul de răspuns interactiv.

3.2 Raportarea la alte soluții

În comparație cu alte platforme, VisioScience3D adresează mai multe nevoi educaționale. În primul rând, ea reprezintă o singură platformă completă pentru toate materiile STEM (în timp ce Phet sau Invatamate se concentrează mai mult pe un subiect, VisioScience3D oferă un mediu unificat cu module interactive de matematică, fizică, chimie, informa-



Figura 3.5: Simulări de pe platforma iqboard

tică și chiar astronomie, reducând necesitatea schimbării constante de aplicații). De asemenea, alt avantaj e reprezentat de simulări 3D în timp real și configurabile (Mozaweb de exemplu are mult suport pentru scene 3D, dar ele sunt predefinite și nu pot fi adaptate dinamic). VisioScience3D permite ajustarea parametrilor în timp real, precum forțele, unghiurile, constantele și animarea vectorilor la fizică. Un alt avantaj major este panoul de monitorizare a progresului (lipsa rapoartelor este o problemă la majoritatea soluțiilor gratuite). Profesorii pot vizualiza instantaneu statistici legate de scoruri, dificultăți întâmpinate și pot adapta testele în funcție de nevoile elevilor. În plus, sistemul de gamificare din VisioScience3D adaugă un plus de interactivitate și dă motivație elevilor (majoritatea platformelor discutate nu au un sistem de gamificare bine definit).

3.3 Motivația alegerii VisioScience3D

VisioScience3D răspunde foarte bine nevoilor profesorilor și elevilor de azi. Oferă simultan simulări 3D pentru toate disciplinele STEM, configurabile direct în browser în platformă, fără pluginuri externe. În timp ce multe platforme existente fie acoperă doar un număr limitat de subiecte, fie implică licențe costisitoare sau dependențe externe (Mozaweb, IQBoard). VisioScience3D pune la dispoziție un mediu unic, adaptat programei românești. Profesorii pot monitoriza progresul elevilor prin dashboard-uri integrate și primesc rapoarte detaliate. Prin modelul gratis și nivelul simulărilor suportate, VisioScience3D este un ecosistem complet, flexibil și orientat spre rezultatele elevilor.

Capitolul 4

Tehnologii utilizate pentru front-end

4.1 Soluția de front-end

4.1.1 Framework principal

Frameworkul principal care s-a folosit este React JS², care este o tehnologie foarte populară și răspândită bazată pe JavaScript³, una dintre cele mai populare limbaje de programare destinate web-ului existent. În alegerea de tehnologie pentru front-endul platformei VisioScience3D, am concentrat atenția pe criterii precum maturitatea ecosistemului, performanță, curba de învățare, flexibilitate în personalizare și capacitatea de integrare cu biblioteci 3D. Dintre principalele opțiuni (Vue.js, Angular, Svelte), React s-a impus datorită următoarelor avantaje:

Ecosistem matur și susținere mare: React are o comunitate activă de milioane de dezvoltatori, cu un număr impresionant de librării, tutoriale și un suport consistent din partea Meta. Această resursă ne-a permis să adoptăm rapid bune practici și să găsim soluții la provocări specifice dezvoltării 3D.

Model declarativ și component-driven: React oferă un mod de declarare detaliată în descrierea interfeței, ideal pentru scene 3D complexe, unde starea se propagă predictibil prin componente. Aceasta ușurează managementul ciclului de viață al obiectelor din biblioteca 3D, reducând riscul de actualizări inconsistente.

React Router DOM: Pentru că oferă o soluție de rutare foarte ușoară în navigarea între diferitele secțiuni ale platformei (fizică, chimie, astronomie etc.) prin React Router

²<https://reactjs.org/>

³<https://www.javascript.com/>

DOM . Acesta face mai simplă definirea rutelor și suportul pentru încărcare întârziată de module. Alternativele (Vue Router, Angular Router) sunt la fel de capabile, însă integrarea lor cu React Three Fiber, care vom vedea este baza în simulările 3D, ar fi impus un strat suplimentar de interoperabilitate.

Pe zona de stilizare a interfeței, am optat pentru Tailwind CSS¹, un framework CSS utilitar care ne-a permis să creăm un design personalizat. Am comparat framework-uri clasice de CSS (Bootstrap, Material UI) cu Tailwind CSS și am observat următoarele avantaje:

Prioritatea pe utilitare și JIT²: Tailwind oferă clase utilitare care permit definirea rapidă a layout-urilor și a stilurilor unice, fără a scrie sutele de linii de CSS personalizat. Motorul JIT (Just-In-Time) generează doar clasele folosite, menținând pachetul final mai mic ca dimensiune.

Consistență și flexibilitate: În loc să se încărce aplicația cu componente predefinite, s-a putut construi o interfață în mod granular respectând paleta de culori și spațiile dorite, dar fără a rescrie componente UI complexe.

Integrare bună cu React: Utilizarea `className` din JSX se potrivește în mod natural cu framework-ul Tailwind. Mai ales plugin-urile pe care le oferă precum `@tailwindcss/forms` ajută configurarea unui stil constant pentru toate formularele.

Extensibilitate: Tailwind permite personalizarea simplă a temelor, adăugarea ușoară de plugin-uri externe și crearea de componente reutilizabile în UI, fără a sacrifica viteza de dezvoltare.

Bibliotecile de creare și gestionare a graficii 3D sunt esențiale pentru platforma noastră, iar pentru a crea simulări interactive, am ales Three.js³, o bibliotecă JavaScript populară. Three.js oferă un API bine gândit pentru crearea de scene 3D, animații și interacțiuni și împreună cu React Three Fiber (R3F)⁴, o bibliotecă care integrează Three.js cu React, am putut să ne concentrăm pe dezvoltarea rapidă a aplicațiilor 3D fără a ne preocupa de detaliile de implementare ale Three.js. Utilizarea peste a Drei⁵, ce conține o suită de componente și utilitare pentru R3F, a permis accelerarea procesului de dezvoltare, oferind soluții gata făcute pentru anumite probleme comune.

Câteva dintre beneficiile integrării cu Three.js prin React Three Fiber și Drei care au

¹<https://tailwindcss.com/>

²<https://tailwindcss.com/docs/just-in-time-mode>

³<https://threejs.org/>

⁴<https://r3f.docs.pmnd.rs/r>

⁵<https://drei.pmnd.rs/>

făcut alegerea justificată sunt:

Abstracție declarativă: R3F încapsulează API-ul Three.js într-un DSL¹ React, potențând definirea de scene, obiecte și materiale în JSX², sub formă de componente. Astfel, putem folosi hook-uri (useFrame, useLoader) pentru a sincroniza animațiile și resursele fără prea mult cod duplicat.

Ecosistem Drei: Biblioteca Drei aduce peste 100 de componente existente (OrbitControls, Text, etc.), accelerate de comunitate, care au economisit timp de implementare.

Optimizări de performanță: Datorită reconciler-ului React, R3F actualizează doar părțile din scenă care se schimbă.

Așadar, combinația React + React Router DOM + Tailwind CSS + React Three Fiber + Drei ne-a oferit un stack unificat, performant și ușor de întreținut, bine adaptat pentru dezvoltarea rapidă de simulări 3D interactive în browser. Se poate observa în figura de mai jos cum arată arhitectura simplificată a frontend-ului platformei.

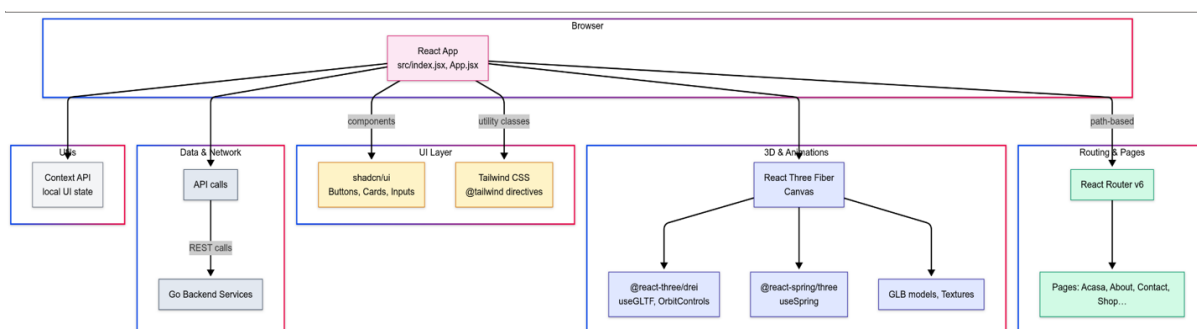


Figura 4.1: Diagrama de design principală a frontend-ului

4.1.2 Framework 3D

După cum am discutat anterior, frameworkul 3D ales pentru acest proiect este React Three Fiber (R3F). Această bibliotecă integrează Three.js cu React, permițându-ne să creăm scene 3D într-un mod declarativ și reactiv. R3F combină principiile WebGL cu capacitățile oferite de React, precum componentizarea și gestionarea stării.

Unul dintre avantajele principale ale R3F este că ne permite să integrăm hook-uri, care îmbunătățesc eficiența și interactivitatea aplicației. De exemplu, hook-ul useFrame permite actualizarea scenei la fiecare cadru, ajutând animațiile și interacțiunile în timp real. UseLoader este folosit pentru încărcarea resurselor externe, precum texturi

¹Limbaj specific de domeniu

²<https://react.dev/learn/writing-markup-with-jsx>

sau modele 3D, într-un mod eficient, prin gestionarea încărcării asincrone folosind promisiuni.

Un alt hook important este `UseThree`, care oferă acces direct la obiectul `Three.js` curent, permițându-ne să interacționăm cu scena, camera și renderer-ul. De asemenea, `useRef` este util pentru crearea de referințe la obiectele din scenă, facilitând manipularea acestora în timpul animațiilor sau interacțiunilor.

Pentru gestionarea stării componentelor, folosim `useState`, care permite actualizarea și redarea dinamică a obiectelor din scenă. În plus, `useEffect` ne ajută să gestionăm efectele secundare, cum ar fi adăugarea de evenimente sau actualizarea stării în funcție de modificările din scenă.

Pentru integrarea obiectelor cu extensia `.glb` am folosit un convertor GLTF¹ care ne permite să convertim modele `.glb` în componente `React Three Fiber` care conțin grupuri de meshe-uri și materiale. Acest lucru a permis importarea modelelor 3D direct în aplicația `React` și crearea de diverse manipulări și animații folosind API-ul `R3F`.



Figura 4.2: Exemplu de convertire a unui model 3D `.glb` în `R3F`

4.1.3 Modelare 3D

Am folosit `Blender` pentru a crea modelele 3D utilizând modele și primitive existente pe internet. De exemplu modelul paginii principale a fost făcut din primitive de insule și castele și adaptat la un număr de insule potrivit materiilor din meniu, cât și loc pentru suport viitor pentru alte materii.

¹<https://gltf.pmnd.rs/>

Putem observa în figura următoare o scenă din dezvoltarea modelului în Blender.

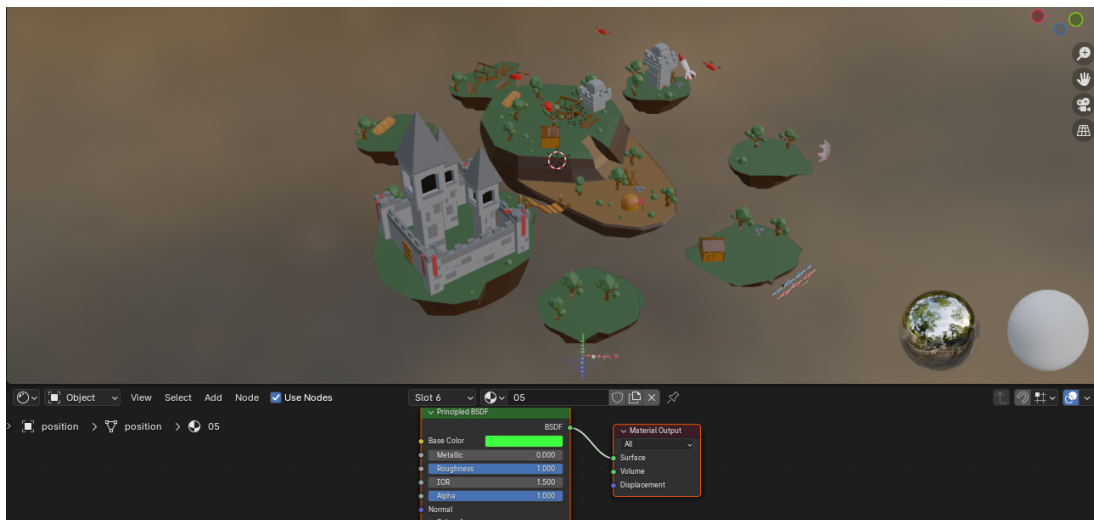


Figura 4.3: Modelul paginii principale în Blender

La acest model s-au adăugat animații de rotație și schimbare a state-ului la mișcarea mouse-ului, animație continuă de plutire și animații realiste de zbor pentru dronele din jurul insulelor.

4.2 Soluția UI/UX

4.2.1 Branding-ul proiectului

Paleta de culori

Brandingul VisioScience3D se bazează pe o paletă caldă, prietenoasă și destul de pastelată, menită să creeze o atmosferă de gamificare pentru elevi. Fundalurile folosesc un gradient simplu de la #fdf4ff (alb-roz pal) prin #f3e8ff (lavandă) spre #fff7ed (piersică pal). Culoarele de accent dau personalitate interfeței, cu nuanțe de mulberry (#690375) ce marchează elementele active, bordurile și textele importante și cu tonul maro rozaliu (#AE847E) ce individualizează secțiunile de chimie și astronomie. Pentru evidențierea vizualizărilor 3D s-a introdus un violet închis (#4f46e5). Culoarele reflectă o atmosferă dinamică și energetică, încurajând explorarea și învățarea.

Logo-ul

Logo-ul VisioScience3D este relativ simplu și în temă cu paleta de culori, fiind reprezentat de cuvântul "VisioScience3D" scris cu un font reprezentativ și colorat cu un

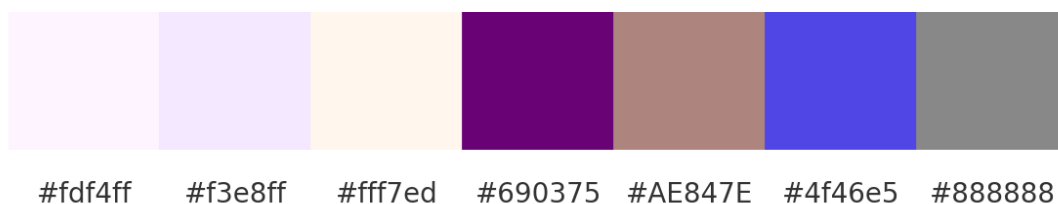


Figura 4.4: Paleta de culori a platformei

gradient de la mov la mulberry (#690375).

VisioScience3D

Figura 4.5: Logo-ul platformei

Mascota platformei

Mascota proiectului este un balon care se apropie de paleta de culori a platformei și care simbolizează explorarea și învățarea. Balonul apare în toate paginile importante ale platformei și este de fiecare dată animat potrivit acțiunilor paginii respective. De exemplu, pe pagina de înregistrare mascota se mișcă în sus și în jos când utilizatorul scrie în câmpurile de înregistrare, iar la înregistrare reușită mascota sare în sus și în jos pentru câteva secunde. Balonul mascotă este și caracterul de explorare a meniului principal, învârtindu-se între insule pentru a ajunge la diferitele materii.



Figura 4.6: Mascota platformei

4.2.2 Experiența utilizatorului

Navigarea în platformă

Navigarea în platformă este realizată printr-un meniu principal situat în partea de sus a paginii, iar în meniul 3D prin rotirea balonului mascotă în jurul insulelor pentru a ajunge la diferitele materii. În profil meniul funcționează pe bază de stări și se schimbă în funcție de pagina curentă.

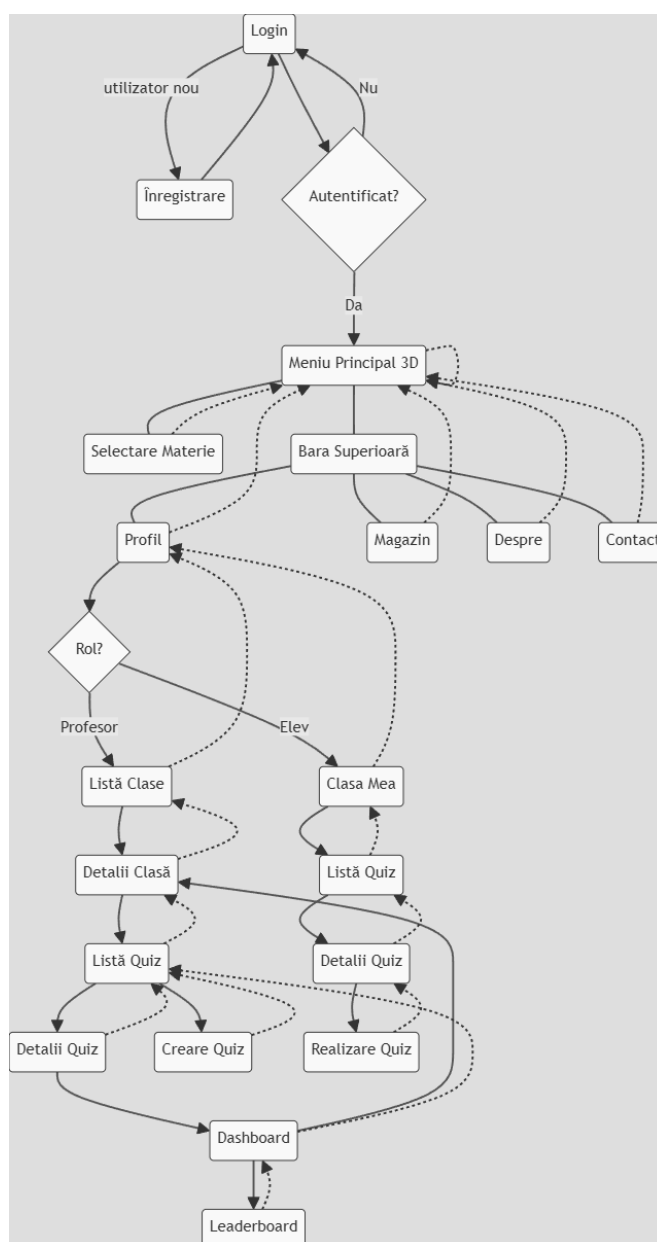


Figura 4.7: Diagrama de navigare a platformei

Gestionarea stărilor

Gestionarea stărilor legate de autentificare și profilul utilizatorului se realizează prin stări locale (`useState`), efecte de ciclu de viață (`useEffect`) și persistență în `localStorage`:

- **Stări locale:** variabilele declarate cu `useState` păstrează răspunsul API-ului și orice mesaj de eroare.
- **Efecte asincrone:** în `useEffect` se trimite cererea HTTP cu token-ul extras o singură dată din `localStorage`, la succes se apelează `setUser()`, iar la eșec `setError()`. Indiferent de rezultat, în `finally` se setează `setLoading(false)`.
- **Persistență:** token-ul JWT și `userId` sunt citite și stocate o singură dată de la prima montare a componentei, ceea ce permite menținerea autentificării între reîncărcări.

Acest model este simplu dar s-ar putea extinde cu *React Context* sau biblioteci dedicate (*Redux*, *Zustand*) dacă numărul de componente care împart aceleași stări ar crește semnificativ.

Proiectarea interfeței utilizatorului

Interfața păstrează un design simplu cu meniul plasat sus și în stânga pentru selecția materiilor după rotire la fiecare insulă în particular. Navigarea prin materii reprezintă o rotație de 360 de grade a camerei în jurul insulei principale, iar după o rotație se revine la instanța inițială a castelului insulei principale.

Panoul profilului este simplu și ușor de folosit, având elemente care ies ușor în evidență și care sunt ușor de accesat cu acțiuni simple de tipul selecție, drag-and-drop sau butoane de activare/dezactivare.

4.3 Soluția de creare a scenelor 3D

4.3.1 Utilizarea tehnologiilor WebGL și Three.js

Crearea și gestionarea scenelor 3D

Scenele 3D se creează folosind tagul de tip `Canvas` din *React Three Fiber*, care ne permite să definim o zonă de desenare 3D unde se pot adăuga obiecte, camere și lumini. Acestea sunt gestionate printr-o ierarhie de componente *React*, fiecare reprezentând un obiect 3D sau un grup de obiecte.

Se folosește tagul `a` din `@react-spring/three` care ne dă acces la grupuri de obiecte și materiale, permițându-ne să transformăm obiecte `glb` în `jsx`.

Interacțiunea cu obiectele 3D

Folosim `useFrame` pentru a actualiza scena la fiecare cadru, simulând callback-urile clasice din buclele din jocuri. `UseEffect` e utilizat să actualizeze starea componentelor în funcție de modificările din props, practic folosind `addEventListener` pentru a asculta evenimentele de schimbare a stării pe canvas sau document. `UseRef`, `UseGLTF` și `UseThree` au rolul de a crea referințe la obiectele din scenă, permițându-ne să le manipulăm direct în timpul animațiilor sau interacțiunilor. Folosim callbackuri de bază ca și `stopPropagation` pentru a preveni propagarea evenimentelor mouse-ului de exemplu și `preventDefault` pentru a preveni comportamentele implicite.

Optimizarea performanței graficii 3D

Pentru a optimiza performanța graficii 3D pe framework-ul bazat pe WebGL¹, s-au abordat câteva tehnici generale:

- **Reducerea numărului de draw calls²:** Combinăm obiectele similare folosind `BufferGeometry` pentru a limita costul fiecărui cadru.
- **Frustum culling³ și occlusion culling⁴:** WebGL elimină automat obiectele aflate în afara câmpului vizual, iar React Three Fiber expune aceste mecanisme nativ, astfel încât obiectele invizibile să nu fie trimise GPU-ului.

React Three Fiber (`react-three/fiber`) și Drei (`react-three/drei`) oferă setări și hook-uri dedicate pentru performanță, care sunt esențiale pentru optimizare. De exemplu, setarea `<Canvas dpr=[1,1] />` limitează device pixel ratio⁵ la un interval controlat, prevenind supraîncărcarea GPU-ului pe ecranele cu densitate mare. De asemenea, proprietatea `performance=min:0.5, max:1` ajustează dinamic rata de reîmprospătare a canvas-ului în funcție de activitate, reducând numărul de cadre atunci când scena este statică. Un alt hook important din Drei este `useTexture`, care preîncarcă și cache-uește texturile, evitând alocările repetate și blocările de I/O în bucla de randare.

În proiectul VisioScience3D, aceste principii au fost aplicate în mai multe moduri pen-

¹Framework de bază pentru grafică 3D în browsere, <https://webglfundamentals.org/>

²Draw calls reprezintă apelurile efectuate de CPU către GPU pentru a desena obiecte.

³Tehnică de eliminare a obiectelor care nu sunt vizibile în câmpul vizual al camerei.

⁴Eliminarea obiectelor care sunt blocate de alte obiecte.

⁵DPR reprezintă raportul dintre pixelii fizici și pixelii logici, afectând claritatea graficii.

tru a îmbunătăți performanța aplicației. Texturile au fost decupate și redimensionate la rezoluții de $1k-2k$ și convertite în formate comprimate (JPEG/PNG cu bitrate optim), reducând încărcarea pe GPU. Geometria sferică a planetelor utilizează un număr optim de segmente (32–64), echilibrând netezimea cu numărul de vârfuri procesate. Pentru a evita recrearea geometriilor și materialelor la fiecare rerandare, am folosit `React.memo` și `useMemo`. Animațiile sunt gestionate exclusiv pe GPU prin `useFrame`, asigurându-se astfel un randament mai bun. În plus, setările WebGL `toneMapping` și `outputEncoding` au fost ajustate pentru a profita de hardware-ul mai slab și cel mobil, folosind `powerPreference:low-power`. De asemenea, pentru a optimiza performanța graficii 3D, folosim texturi de dimensiuni mici și medii, pentru a asigura stabilitate și un consum redus de resurse.

4.3.2 Integrarea cu frontend-ul

Pentru integrarea modelelor 3D în platformă, fiecare scenă sau obiect (.glb) este încărcat cu declanșatorul `useGLTF` din `@react-three/drei` și convertit într-o componentă React care reacționează la stările de aplicație. De exemplu, componenta `Baloon` folosește un `useEffect` pentru a schimba periodic `animationState` dintr-o listă de stări (`idle`, `swayLeft`, `floatUp` etc.), iar declanșatorul `useSpring` din `@react-spring/three` transformă această stare într-un set de proprietăți `position` și `rotation` animate, atribuite mai departe unui element `mesh`.

În plus, componenta de background atașează direct evenimente DOM (`mousedown`, `mousemove`, `keydown`) și le gestionează în interiorul `useFrame` din frameworkul de React, permițând controlul direct al rotației scenei în funcție de interacțiunea utilizatorului. Starea și logica aplicației se propagă în lumea 3D, creând o legătură reactivă între UI-ul convențional și grafica gestionată prin WebGL.

Capitolul 5

Tehnologii utilizate pentru back-end

5.1 Descrierea arhitecturii

Backend-ul este construit folosind o arhitectură cu microservicii, care permite scalare și întreținere ușoară. Avem patru servicii principale pentru logică de produs, un serviciu care funcționează ca router al arhitecturii, două instanțe de baze de date nerelaționale, un serviciu de gestionare a infrastructurii, un serviciu de monitorizare care funcționează împreună cu un serviciu de provizionare de metrice și un serviciu de vizualizare de metrice și în final utilitarele corespunzătoare pentru gestionarea bazelor de date integrate.

S-a ajuns la o asemenea arhitectura pe microservicii deoarece o variantă monolit nu răspundea bine nevoilor platformei, care are diferite componente cu scop total separat. Ele comunică foarte puțin între ele, mai mult făcând-o cu frontend-ul printr-un router dedicat. În acest mod, s-a putut dezvolta fiecare parte a backend-ului separat, fără a influența componente deja implementate. Împărțirea pe servicii s-a realizat în funcție de scopul principal, făcând diferența între datele utilizatorilor, datele educaționale, codul C++ din fluxul interactiv și altele.

5.1.1 Configurarea clusterului

Clusterul este configurat folosind Kubernetes², care permite orchestrarea containerelor și gestionarea resurselor print-o soluție larg folosită. Fiecare serviciu este containerizat folosind Docker, permițându-ne să rulăm aplicațiile într-un mediu izolat și consistent. Apoi containerele sunt încărcate în Docker, de unde se poate face lansarea sau oprirea

²<https://kubernetes.io/>

lor în unități logice numite pod-uri. Kubernetes gestionează scalarea automată a serviciilor, distribuția sarcinilor și asigură disponibilitatea acestora.

Alegerea Kubernetes este justificată de specificul platformei dezvoltate, dar și de publicații precum [14], care scoate în evidență avantajele principale ca scalabilitatea crescută, portabilitatea și flexibilitatea pe care o poate aduce în dezvoltarea și implementarea de platforme Web. Pentru dezvoltare rapidă și locală s-a folosit Kubernetes în Docker (kind) care a permis îmbinarea și a avantajelor oferite de Docker, putând lansa container Docker în Kubernetes după cum am menționat. Acest lucru a ajutat și testarea codului înainte de a-l lansa în diferite medii.

Dezvoltarea astfel a permis integrarea rapidă a noilor funcționalități și testarea acestora într-un mediu controlat. De asemenea, a permis simularea unor scenarii de scalabilitate și performanță, asigurând că platforma poate gestiona un număr mare de utilizatori și cereri simultane.

Procesul de implementare și lansare presupune construirea imaginilor Docker¹ pentru fiecare serviciu, încărcarea acestora în registrul de imagini gestionat prin Docker Desktop pentru testare și apoi reîncărcarea acestora în clusterul Kubernetes. Acest proces este automatizat prin intermediul unui pipeline CI/CD în mediul de dezvoltare Github. Pentru asta se folosește GitHub Actions², care permit rularea automată a diferitelor procese de construire, formatare și apoi trecerea prin zona de testare și lansare a aplicației. Practic folosind GitHub Actions, am putut să creăm un flux de integrare și livrare continuă³ care a permis să automatizăm procesul de construire, testare și implementare a platformei.

Fiecare serviciu are propriile fișiere *.yaml*⁴ care definesc după caz modul de rulare, variabilele de mediu, resursele necesare, rutarea, porturile expuse și alte setări specifice.

5.1.2 Structurarea și crearea fișierelor YAML

Pentru fiecare componentă a backend-ului (Deployment, Service, Ingress, ConfigMap, PersistentVolumeClaim) s-au creat fișiere YAML separate în directoarele *.k8s/* ale fiecărui serviciu. Denumirea fișierelor corespunde resursei Kubernetes pe care o gestionează.

¹Tehnologie de containerizare a aplicațiilor software <https://www.docker.com/>

²<https://github.com/features/actions>

³<https://www.redhat.com/en/topics/devops/what-is-ci-cd>

⁴Fișiere de configurare, în limbaj de marcat

S-au folosit `*-deployment.yaml`¹, `*-service.yaml`², `*-ingress.yaml`³, `*-configmap.yaml`⁴ și `*-pvc.yaml`⁵.

Fișierele de Deployment pentru servicii sunt structurate astfel încât să definească proprietățile principale ale serviciului, cum ar fi numele, etichetele, numărul de replici, selectorul de etichete și specificațiile containerelor.

```
1 apiVersion: apps/v1, kind: Deployment
2 metadata: name, labels
3 spec: replicas: 1, selector: matchLabels: app, template:
4   metadata: labels: app, spec: name, image: .., ports, env: ..
```

Acesta este un exemplu de specificații din Deployment-ul pentru unul dintre servicii, cu o singură replică pentru rulare. El specifică imaginea Docker care va fi utilizată. De asemenea, definește variabilele de mediu necesare (ca cele de conectare la baza de date MongoDB).

Tipuri de resurse și rolurile lor

Service (`kind:Service`): expune pod-urile pe un IP intern al clusterului. Astfel, alte servicii pot comunica cu acesta prin numele său și portul specificat. Accesul din exterior nu este permis direct. Astfel se realizează o comunicare internă sigură între servicii, iar accesul exterior e permis prin intrarea unică în Kong Gateway. Putem observa structura fișierului de configurație pentru un acest tip de serviciu.

```
1 apiVersion: v1, kind: Service
2 metadata: name
3 spec: type, selector: app, ports: port: 8080, targetPort: 8080
```

Ingress (`kind:Ingress`): definește rutarea HTTP/S prin controller-ul folosit. Acesta permite expunerea serviciilor către exterior printr-un singur punct de intrare și permite gestionarea traficului prin reguli de rutare, posibile firewalls sau balansarea încărcării pe servicii. Acest Ingress poate fi adăugat apoi în configurarea oricărui alt serviciu. S-au testat ambele variante cu truncare a căii sau fără, dar s-a ales cea fără truncare a căii pentru a permite accesul direct la un serviciu anume printr-un cuvânt cheie în URL. Și varianta cu truncare ar fi funcționat, dar ar fi existat un număr foarte mare de

¹<https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>

²<https://kubernetes.io/docs/concepts/services-networking/service/>

³<https://kubernetes.io/docs/concepts/services-networking/ingress/>

⁴<https://kubernetes.io/docs/concepts/configuration/configmap/>

⁵<https://kubernetes.io/docs/concepts/storage/persistent-volumes/>

rute în rădăcina ruterului Kong, ceea ce ar fi putut duce la probleme de gestionare sau posibile conflicte de nume.

```
1 apiVersion: networking.k8s.io/v1, kind: Ingress
2 metadata: name, annotations: konghq.com/strip-path: "false"
3 spec: ingressClassName, rules: - http: paths: - path, backend:
    ↪ service: name, port
```

ConfigMap¹ & PVC²: ConfigMap este folosit pentru configurări directe (de ex. pentru serviciul de provizionare prometheus.yml) și PVC este folosit de exemplu pentru volumul serviciului de vizualizare a metricilor în Grafana. Putem observa structura de configurare a celor două tipuri adiționale de resurse.

```
1 apiVersion: v1, kind: ConfigMap
2 metadata: name, data: prometheus: global: scrape_int, eval_int
3 ---
4 apiVersion: v1, kind: PersistentVolumeClaim
5 metadata: name, spec: accessModes, res: requests: storage
```

Fiecărei resurse îi corespunde un singur fișier YAML care conține versiunea api-ului, tipul resursei kind, metadatele și specificațiile. Acestea pot fi aplicate cu `kubectl apply -f <fișier>`, sau automatizat printr-un pipeline de integrare continuă / livrare continuă (CI/CD) cum am discutat anterior.

5.1.3 Gestionarea clusterului

Clusterul este creat și gestionat după cum s-a prezentat anterior cu Kubernetes, folosind manifesturi YAML. Pe lângă linia de comandă, am integrat Portainer³, pentru o interfață web de administrare. Portainer rulează ca un Deployment sub serviciul portainer-sa, căruia i s-a atribuit un ClusterRole⁴ cu toate privilegiile și un ClusterRoleBinding aferent. Manifestele corespunzătoare (portainer-sa, portainer-role, portainer-deployment, portainer-rolebinding, service și portainer-ingress) sunt folosite pentru configurare și gestionare a accesului Portainer în cluster. Interfața Portainer, expusă printr-un Service și un Ingress pe ruta /portainer, permite vizualizarea nodurilor, pod-urilor, volumelor și gestionarea configurațiilor (scalare, relivrare, mesaje) fără comenzi complexe, oferind în același timp controale RBAC⁵ și monitorizare centralizată.

¹<https://kubernetes.io/docs/concepts/configuration/configmap/>

²<https://kubernetes.io/docs/concepts/storage/persistent-volumes/>

³<https://www.portainer.io/>

⁴<https://kubernetes.io/docs/reference/access-authn-authz/rbac/>

⁵Controlul accesului bazat pe roluri

Se poate vedea în imaginea de mai jos cum arată interfața Portainer pe clusterul platformei:

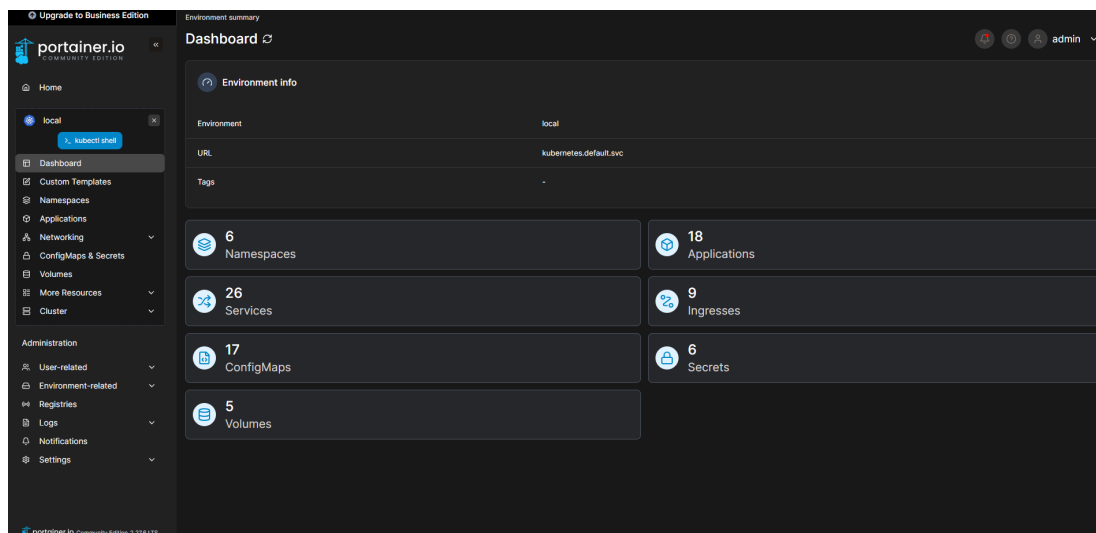


Figura 5.1: Interfața Portainer pe cluster

Si mai jos se poate observa lista de servicii in linie de comandă versus în Portainer:

[illegible]

(a) Serviciile în Portainer

(b) Serviciile în linie de comandă

5.1.4 Rutarea si gestionarea traficului

Pentru a centraliza și controla traficul HTTP/S, am ales Kong¹ ca API Gateway. În cluster s-a definit o clasă de rutare dedicată după configurarea Kong ca serviciu separat și s-a adăugat această clasă în manifestele fiecărui serviciu ce are nevoie de expunere.

Controller-ul Kong va intercepta toate resursele de tip Ingress care specifică numele clasei kong. În mediul de dezvoltare, pentru a nu expune un LoadBalancer² extern, am folosit port-forwarding³ local prin:

```
kubectl port-forward svc/kong-kong-proxy 8000:80
```

¹<https://konghq.com/>

²<https://kubernetes.io/docs/concepts/services-networking/>

³<https://kubernetes.io/docs/tasks/access-application-cluster/port-forward-access-application-cluster/>

Astfel, apelurile HTTP către portul 8000 local sunt tunelate prin Kong și pot fi testate rapid în legătură cu serviciile din cluster fără configurări suplimentare de rețea.

5.2 Descrierea serviciilor

În continuare sunt listate serviciile care alcătuiesc backend-ul aplicației, fiecare având responsabilități specifice în cadrul arhitecturii pe microservicii.

evaluation-service – Serviciul pentru gestionarea logicii de creare, monitorizare, deschide/închidere, ștergere, gestionare de teste și evaluări. Funcționează ca un controller REST API care preia datele din bazele de date, le modelează pentru a fi transmise către frontend și încapsulează și logica de calculare și trimitere către stocarea persistentă de punctaje, scoruri, clasamente și alte statistici.

feed-data – Serviciul care se ocupă de gestionarea datelor legate de secțiunile educaționale și accesarea motorului de generare a moleculelor de chimie. Datele brute din fișierele .mol¹ ajung în acest serviciu și sunt parsate linie cu linie pentru a identifica poziția atomilor, legăturile chimice dintre ele, tipul lor, dimensiunile și tipul moleculelor. Datele sunt apoi transmise către routerul serviciului pentru a fi accesibile prin API-ul REST către frontend.

cpp-compiler-service – Serviciul care încapsulează logica de gestionare a compilării codului C++ inserat în editorul interactiv și îl împarte în capturi de status a structurilor de date ce apar. Codul este primit din frontend și trece prin mai multe etape de procesare. Se trece linie cu linie prin el pentru a identifica structurile de date folosite și a le extrage. Al doilea pas este de creare și adăugare a unui antet de urmărire a fiecărei structuri de date capabil să înregistreze și raporteze statusul la fiecare operație. La ultimul pas aceste statusuri sunt adnotate cu metadatele necesare și inserate cronologic într-o listă de evenimente ca să fie trimisă apoi către frontend.

grafana – Utilitar pentru vizualizarea metricilor și a datelor de monitorizare. Practic este gazdă pentru rularea imaginii de Grafana² care primește metricile colectate de Prometheus³ și le afișează într-o interfață web.

kong-kong – Serviciul găzduiește pentru controller-ul Kong de gestionarea a rutării și a API-urilor despre care s-a discutat anterior.

¹Fișiere de descriere a moleculelor chimice

²<https://grafana.com/>

³<https://prometheus.io/>

`mongo-express-feed-data` – Interfață web pentru gestionarea bazei de date a feed-urilor educaționale. Oferă o interfață grafică pentru vizualizarea și manipularea datelor din baza de date MongoDB asociată cu serviciul de feed-uri.

`mongo-express-user-data` – Interfață web pentru gestionarea bazei de date a utilizatorilor. Oferă ca și cea anterior menționată o interfață grafică pentru vizualizarea și manipularea datelor din baza de date MongoDB asociată cu serviciul de utilizatori.

`mongo-feed-data-service` – Baza de date MongoDB pentru serviciul de feed-uri educaționale.

`mongo-user-data-service` – Baza de date MongoDB pentru serviciul de utilizatori.

`monitoring-service` – Serviciul de monitorizare care colectează și expune metrice. Practic aici se încapsulează părțile de cod care folosesc instrumentarea oferită de Prometheus pentru a crea interfețe reutilizabile ce vor fi inserate în zonele din celelalte servicii unde se dorește colectarea de metrice.

`portainer` – Interfață de administrare a clusterului Kubernetes. Oferă toate operațiile posibile pentru operarea cu resursele din cluster.

`prometheus` – Serviciul care este gazdă pentru imaginea de Prometheus¹ folosită în serviciul de monitorizare.

`user-data` – Serviciul pentru gestionarea datelor utilizatorilor, inclusiv autentificare și autorizare. El funcționează ca un router REST API care are mai multe straturi prin care trec datele. Primul este cel de validare a token-ului de autentificare în middleware-ul JWTAuth, apoi prin parsarea datelor, stocarea lor persistentă în baza de date MongoDB și apoi prin reexpunerea lor către frontend prin API-ul REST.

Mai jos în figura 5.2 putem observa și arhitectura principală de comunicare între serviciile din back-end.

5.2.1 Descrierea API-urilor

Fiecare serviciu are un API RESTful care permite interacțiunea cu resursele sale. Aceste API-uri sunt definite în routerul fiecărui serviciu care este creat prin biblioteca Gorrilla Mux în Go. Fiecare serviciu are un set de endpoint-uri care permit operații CRUD (Create, Read, Update, Delete) asupra resurselor sale și interacționează cu baza de date corespunzătoare.

¹<https://prometheus.io/>

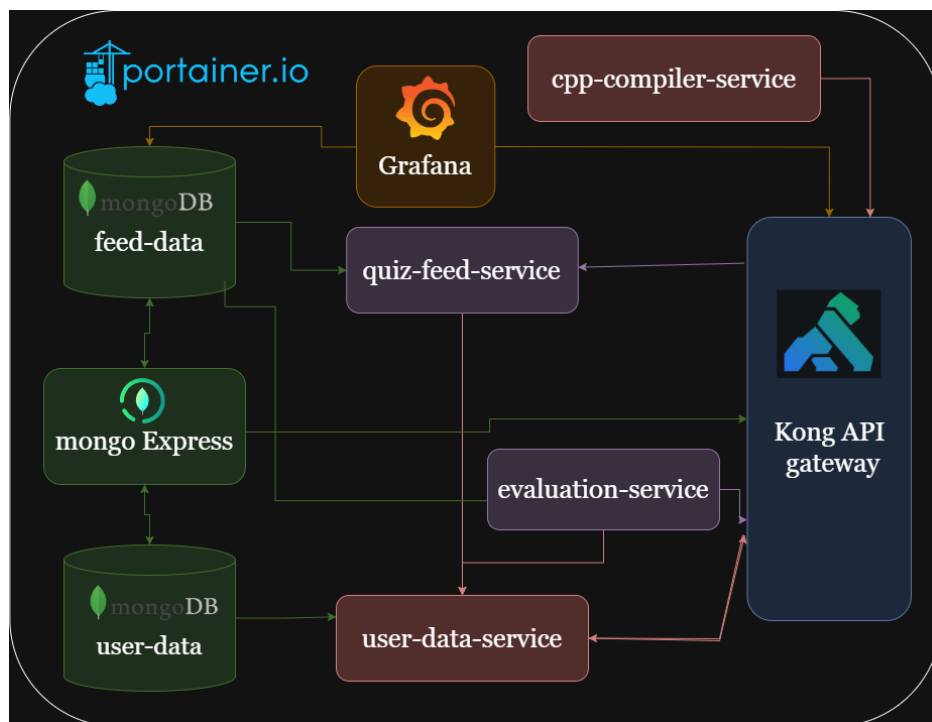


Figura 5.2: Arhitectura backend-ului aplicației

5.3 Securitate

Toate API-urile serviciilor sunt protejate prin JWT¹, validat de secțiunea JWTAuth, care respinge cererile fără token sau cu token invalid. Traficul extern este dirijat prin Kong după cum am văzut, configurat cu HTTPS/TLS, politici de limitare a numărului de cereri HTTP, CORS² și trunchere a căii pentru a preveni atacurile de tip „path traversal”³ și abuzul de resurse. În interiorul clusterului, rolurile și permisiunile sunt gestionate prin RBAC Kubernetes (ClusterRole/ClusterRoleBinding⁴), iar Portainer rulează sub un ServiceAccount cu drepturile strict necesare la acces. Conexiunile la bazele MongoDB sunt realizate folosind URL-urile securizate (MONGO_URI) și logare cu credențiale la serverul de baze de date.

¹JSON Web Token, folosit în protocoale de autentificare

²Cross-Origin Resource Sharing

³Atacuri bazate pe manipularea căii de acces a resurselor

⁴<https://kubernetes.io/docs/reference/access-authn-authz/rbac/>

5.4 CI/CD

5.4.1 Port forwarding

În mediul de dezvoltare, pentru a testa rapid Ingress-ul Kong fără LoadBalancer extern, se folosește port-forwarding după cum am discutat la capitolul Rutarea și gestionarea traficului. Acest lucru permite accesul la serviciile din cluster. Ca pipeline-ul din GitHub Actions să aibă acces la clusterul Kind local s-a folosit un runner GitHub hostat local¹. Acesta permite direct accesul la comenzile de build și push în Docker și mai ales la cele de redeploy al serviciilor în cluster.

5.4.2 Deployment

După cum am menționat am automatizat build-ul, testarea și deploy-ul în Kubernetes cu GitHub Actions.

- **Checkout & Context:** se selectează codul pentru fiecare serviciu și se setează contextul kubectl la kind-visioscience, clusterul unde rulează local mediul de dezvoltare.
- **Build & Push Docker:** se compilează binarele Go, se construiește imaginea cu build-push-action și se încarcă pe DockerHub sub numele <user>/\$SVC:latest.
- **Deploy:** rulăm:

```
1 kubectl apply -f k8s/  
2 kubectl rollout status deployment/extract.outputs.name
```

- Runner-ul GitHub este un hostat pe aceeași mașină ca și clusterul Kind, astfel că are acces direct la kubeconfig².

5.4.3 Metrici / Monitorizare

5.4.4 Avantajele folosirii Prometheus

Prometheus este un sistem de monitorizare și alertare foarte popular în mediile moderne de dezvoltare și producție datorită scalabilității și flexibilității sale. Oferă o arhitectură simplă bazată pe colectarea activă a metricilor prin intermediul unor endpoint-uri HTTP expuse de aplicații, facilitând integrarea rapidă cu o varietate largă de servicii

¹<https://docs.github.com/en/actions/hosting-your-own-runners/about-self-hosted-runners>

²<https://kubernetes.io/docs/concepts/configuration/organize-cluster-access-kubeconfig/>

și microservicii. În plus, arhitectura sa detașată de stare (stateless) și suportul pentru stocarea pe termen scurt cu export către sisteme externe îl fac ideal pentru arhitecturi dinamice, precum în Kubernetes. De asemenea, se integrează nativ cu instrumente vizuale precum Grafana, oferind dashboard-uri ușor de configurat și interpretat de către dezvoltatori. Grafana e o soluție ideală pentru vizualizarea metricilor colectate de Prometheus, fapt menționat și în publicații precum [17], care îl descrie ca o necesitate esențială în platformele moderne.

5.4.5 Colectare Prometheus

În fiecare serviciu Go am folosit clientul oficial Prometheus:

github.com/prometheus/client_golang/prometheus pentru definirea contoarelor, histogramelor și gauge-urilor.

github.com/prometheus/client_golang/prometheus/promhttp pentru expunerea endpoint-ului `/${SERVICIU}/metrics`.

De exemplu, în `evaluation-service` în codul responsabil de metrice definim astfel.

```
1 registry := prometheus.NewRegistry()
2 reqTotal := prometheus.NewCounterVec(...)
3 registry.MustRegister(reqTotal, reqDuration, activeEvals)
4 http.Handle("/eval/metrics", HandlerFor(registry, opt))
```

5.4.6 Grafana Dashboard

De exemplu după ce Prometheus trece prin `evaluation_service_http_requests_total`, se crează un dashboard persistent prin volumele Kubernetes. Dashboard-ul Grafana are în general două tipuri de panouri pentru metrice similare. De exemplu pentru metrica de cereri și evaluări active din serviciul de evaluare:

Timeseries (serie temporală de date) pentru `eval_svc_req_total`.

Gauge (serie de valori) pentru `eval_svc_active_evals`.

Iar apoi ele sunt definite și legate în configurația panoului din Grafana.

```
1 "panels":
2   {"type":"timeseries",
3     "targets":[{"expr":"eval_svc_req_total"}],
4     "title":"HTTP Total Requests"}, ....
```

Aceste panouri sunt importate în Grafana în Evaluation Service Dashboard și actualizate la fiecare 5s (configurabil).

Așadar există trei panouri principale în Grafana pentru trei servicii de logică de produs existente:

Evaluation Service Dashboard pentru evaluation-service

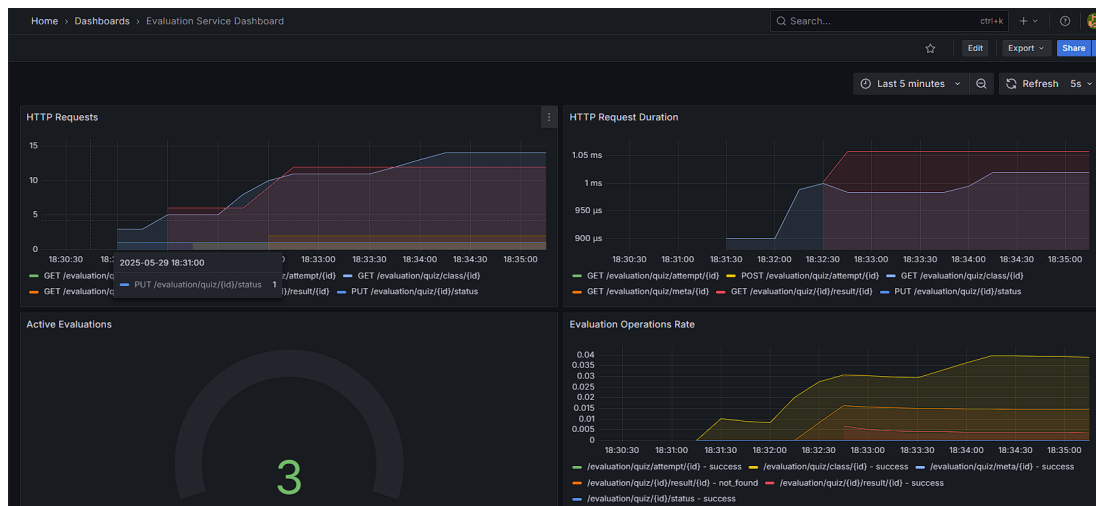


Figura 5.3: Dashboard-ul pentru evaluări în Grafana

Se poate observa logarea numărului de cereri HTTP din serviciu, latența și durata medie a lor, numărul de evaluări active și rata operațiilor pe evaluări.

Feed Data Service Dashboard pentru feed-data

În panou se pot vedea numărul de cereri HTTP, latența și durata medie a lor, numărul de feed-uri active, numărul de molecule active, rata operațiilor asupra feed-urilor și moleculelor, precum și numărul de molecule generate. La fel ca la cel de evaluări, metricile se împart în cele două categorii discutate.

User Data Service Dashboard pentru user-data

Ca și în celelalte panouri, și aici apare numărul, latența și durata cererilor HTTP. Metricile specifice serviciului sunt numărul utilizatorilor și al claselor active.

5.5 Soluția de baze de date

5.5.1 Structura bazei de date

Tipuri de baze de date utilizate / Motivația alegerii

Baza de date folosită este MongoDB¹, o bază de date nerelațională care oferă flexibilitate și scalabilitate și e ușor de folosit și mai ales de integrat în microserviciile gestionate în Kubernetes. Am ales o soluție nerelațională deoarece datele noastre sunt semi-structurate și nu necesită relații complexe între tabele, permițându-ne să stocăm documente JSON care pot fi ușor extinse și modificate fără a necesita migrări de baze de date. Structura se putea să fie integrată și într-un mediu relațional, dar am considerat că MongoDB oferă o flexibilitate mai mare pentru tipurile de date pe care le gestionăm, cum ar fi feed-urile educaționale, datele utilizatorilor, cât și pentru structura testelor și evaluărilor.

Avantajele folosire MongoDB sunt descrise și în publicații precum [15], care menționează flexibilitatea schemelor și pe de altă parte scalabilitatea și performanța pe care o oferă ca avantaje majore. Având în vedere nevoile platformei dar și avantajele menționate și aici și în articolul dat, MongoDB este justificat o alegere bună în cazul proiectului de față.

Un alt avantaj al folosirii MongoDB este că permite stocarea datelor într-o formă orientată pe documente similare cu JSON. Alegerea s-a bazat și pe flexibilitatea schemelor, scalabilitatea orizontală și compatibilitatea nativă cu structurile de date folosite, precum și pe posibilitatea de a face agregări complexe și de a gestiona relații parțiale între documente.

Gestionarea datelor

Datele sunt organizate în colecții care corespund entităților sistemului: quiz-uri, formule, molecule, utilizatori, clase, invitații, etc. Fiecare colecție conține documente ce respectă structurile definite în codul backend-ului, folosind driverul oficial MongoDB pentru Go (go.mongodb.org/mongo-driver).

De exemplu, un document din colecția quizzes conține câmpuri precum Title, ClassID și OwnerID (reprezentate prin id de obiect din MongoDB), un array de Questions cu întrebări, răspunsuri și punctaje, și un array QuizResults pentru stocarea scorurilor utilizatorilor.

¹<https://www.mongodb.com/>

Tabelele sunt separate încât să aibă minimă interacțiune între ele, relaționând maxim prin câteva câmpuri, de obicei id-urile obiectelor din altă colecție. De exemplu, colecția quizzes are câmpul ClassID care face referire la colecția classes, iar colecția users are câmpul ClassID pentru a lega utilizatorii de clasele lor. Arhitectura principală a tabelor din cele două baze de date folosite se poate observa în figura următoare.

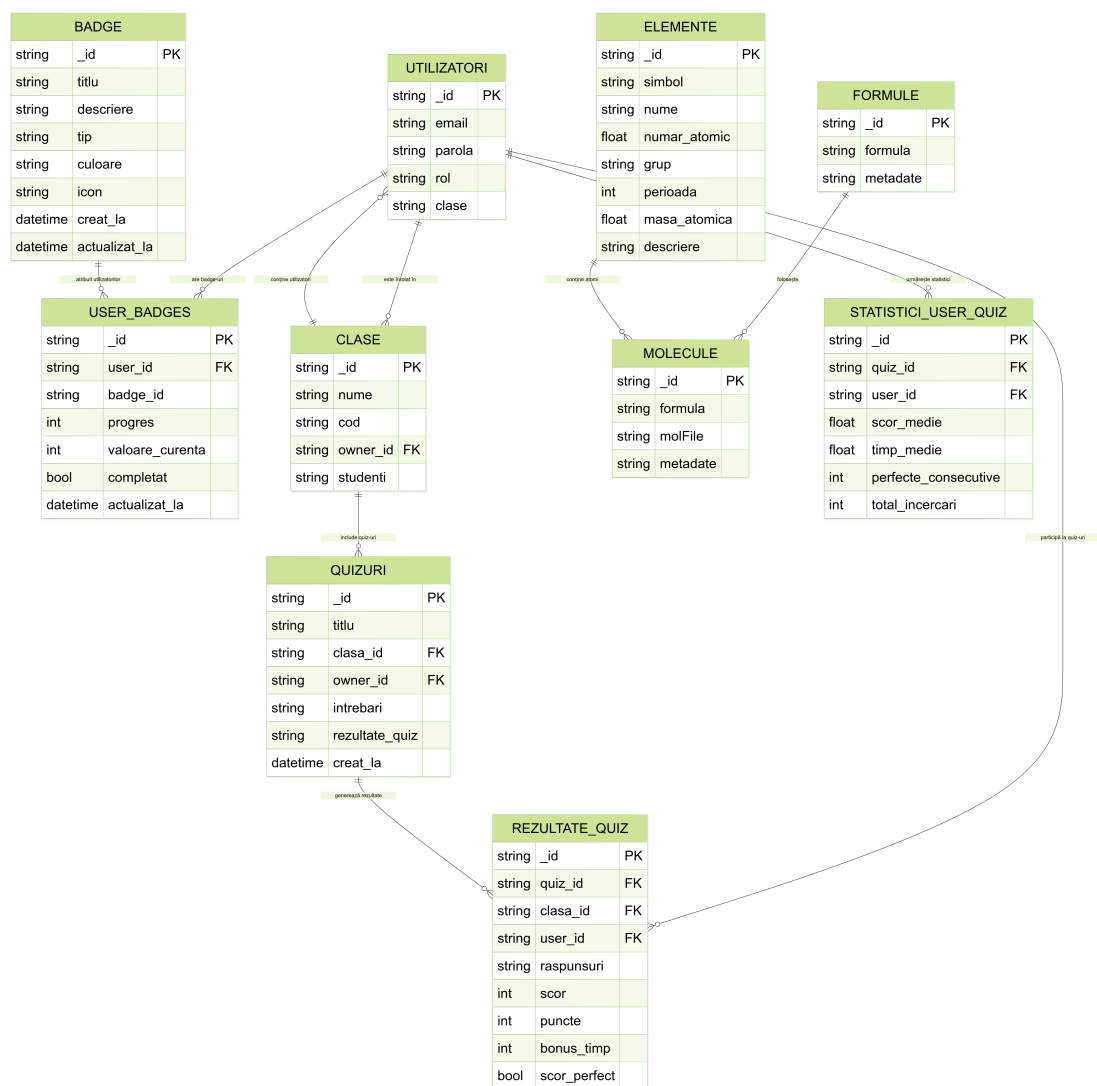


Figura 5.4: Structura principală a tabelor din MongoDB

Se folosesc structuri Go care reflectă aceste modele, permițând deserializarea și serializarea automată între BSON¹ și JSON. Astfel, interacțiunea cu baza de date se face prin operații CRUD folosind metodele MongoDB standard, iar aplicația transformă și face validarea datelor ușor. O problemă întâmpinată des a fost de confundare a

¹<https://www.mongodb.com/resources/basics/json-and-bson>

numelui unui câmp la serializare sau deserializare, de obicei între numele lui Camel-Case și snake_case, dar s-a rezolvat prin implementarea unui strat de uniformizare a structurilor.

Gestionarea și vizualizarea datelor în mediul de dezvoltare a fost făcută folosind interfețele oferite de Mongo express, care permit toate operațiile cu documentele din colecțiile MongoDB printr-o interfață web simplă și intuitivă.

5.5.2 Securitatea datelor

Datele stocate sunt protejate prin controlul accesului la nivel de cluster MongoDB, autentificare și autorizare prin roluri definite (de exemplu, utilizator, profesor). Parolele sunt criptate și gestionate prin variabile de mediu la nivelul serviciilor. Pe viitor se poate face integrarea cu un serviciu de autentificare centralizat (ex: OAuth2, OpenID Connect) sau folosirea unui serviciu de identitate și secrete (precum HashiCorp Vault sau AWS Secrets Manager) pentru a gestiona securitatea și accesul și mai granular la datele sensibile.

Performanța și scalabilitatea bazei de date

MongoDB permite scalarea orizontală prin sharding¹, facilitând distribuția datelor pe mai multe noduri când cantitatea datelor ar ajunge la dimensiuni mari. Indecșii sunt definiți pe câmpuri critice (ex: _id, class_id) pentru a optimiza căutările și interogările.

De asemenea, conexiunile sunt gestionate eficient prin pool-uri în clientul Go, iar operațiile lungi sunt efectuate cu timeout-uri pentru a evita blocarea serviciilor sau cu reîncercare pentru resursele vitale.

5.5.3 Integrarea cu back-endul

Back-endul este implementat în Go, folosind pachetul oficial MongoDB driver, care oferă suport complet pentru operațiile pe colecții și documente, tranzacții și agregări.

La pornire, fiecare serviciu inițializează o conexiune persistentă la baza de date. Modelele Go definite permit maparea clară între structurile din cod și documentele mongo. De exemplu, în evaluation-service, obiectele Quiz și QuizResult sunt manipulate direct în cod, iar metodele expun un API REST pentru aceste resurse prin CRUD².

¹<https://www.mongodb.com/docs/manual/sharding/>

²Create, Read, Update, and Delete

Capitolul 6

Detalii de implementare

În această secțiune se va discuta despre implementarea platformei, acoperind back-endul, front-endul, configurarea, dezvoltarea și conectivitatea acestora.

6.1 Back-end

Pentru back-end s-au analizat diferite tehnologii. Alegerea lor s-a bazat pe comparația descrisă mai jos. Alegerea a fost influențată de mai mulți factori, printre care se numără performanța, ușurința de utilizare și suportul extins pentru dezvoltarea Web . Go oferă un model de concurență eficient și o sintaxă simplă, ceea ce îl face potrivit pentru construirea rapidă de aplicații scalabile. Alt motiv principal a fost compatibilitatea cu infrastructura aleasă pentru proiect, bazată pe Kubernetes și Docker, care se integrează foarte ușor cu Go. Articole precum [13] subliniază popularitatea actuală și avantajele față de alte limbaje precum Node.js sau Python, menționând și punctele susținute anterior. În documentația actuală se prezintă și cazurile de utilizare comune ([12]), printre care principal este dezvoltarea Web.

Pe lângă Go, pentru serviciul de compilare a codului C++ s-a optat pentru implementarea antetului care este inserat pentru detectarea structurilor de date și pentru a crea raportări de status tot la C++ pentru a asigura integrare nativă cu codul primit de la utilizator.

```
1 template<typename Container>
2 class ContainerTracer: public Traceable
3     ContainerTracer(n, c): name(n), container(c)
4         globalTrc->registerObj(this)
```

```

5 class Traceable
6     virtual ~Traceable(): globalTrc->unregisterObj(this)

```

Listing 6.1: Antetul pentru raportarea operațiilor în C++

Se poate observa că antetul se bazează pe structuri generice din C++ pentru a permite orice tip de container să fie integrat în acest tip general de raportare a operațiilor și conținutului.

Tabela 6.1: Compararea limbajelor de programare potrivite pentru VisioScience3D

Caracteristică	Go ¹	Node.js (JavaScript) ²	Python ³
Performanță	Compilat, foarte rapid, cu suport nativ pentru concurență eficientă (goroutine)	Interpretat, bun pentru I/O non-blocant, dar mai lent decât Go	Interpretat, mai lent, CPU-intensive tasks pot fi mai lente
Concurență	Goroutines ușor de folosit, eficiente din punct de vedere al resurselor	Model event-loop, asincron cu callback-uri/promises/async-await	Suport limitat (threading, asyncio), mai complex pentru concurență
Simplitate și claritate	Sintaxă simplă, tipuri de date clare, cod ușor de întreținut	Flexibil, dar poate duce la cod greu de urmărit	Sintaxă clară, dar structurile mari pot deveni greu de gestionat
Ecosistem	În creștere rapidă, multe biblioteci pentru zona de microservicii	Foarte matur și extins, multe biblioteci, framework-uri	Destul de matur, multe biblioteci pentru ML, backend, dar nu neapărat pentru performanță
Scalabilitate	Foarte scalabil, folosit în microservicii și sisteme distribuite	Scalabil pe I/O, dar single-threaded cu limite pe CPU	Scalabilitate limitată, se bazează pe external scaling și caching
Deploy și containerizare	Binar unic compilat, ușor de livrat în containere	Necesită runtime Node.js, imagini mai mari	Necesită runtime Python și biblioteci, imagini relativ mari

Fiecare microserviciu are propria configurație Kubernetes definită prin fișiere YAML pentru Deployment, Service, ConfigMap și alte resurse necesare. Configurarea constă în containerizarea și automatizarea livrării folosind Docker pentru împachetarea imaginilor serviciilor și GitHub Actions pentru lanțul CI/CD, care automatizează construirea, testarea, și deploy-ul pe clusterul Kubernetes local.

Configurarea conexiunilor la baza de date MongoDB este centralizată în secțiuni ajutoare, folosind driver-ul oficial MongoDB pentru Go. Monitorizarea serviciilor este asigurată prin integrarea cu Prometheus, cu metrice personalizate definite în cod și

expuse la endpoint-uri REST. Metricile sunt incrementate direct în cod, cum se poate vedea și în exemplul de mai jos.

```
1 func prometheusMiddleware(next http.Handler) http.Handler
2     return http.HandlerFunc = func(writer w, req r)
3         if path == "/feed/metrics" then serve(w, r), return
4         start := timer(metrics.labels(r.Method, r.path))
5         next.ServeHTTP(w, r), start.ObserveDuration()
6         metrics.reqTotal.labels(r.Method, r.path, "200").Inc
```

Listing 6.2: Middleware Prometheus pentru monitorizarea duratei cererilor HTTP

6.1.1 Dezvoltare back-end

Codul este organizat în pachete separate pentru rute, modele de date, pachet cu fișiere ajutătoare, metrici, utils, panou Grafana, pachet cu fișiere Kubernetes de configurare și middleware (ex: autentificare JWT).

Pentru rutare se utilizează biblioteca Gorilla Mux, iar fiecare endpoint implementează operații CRUD pentru toate resursele din platformă precum quizuri, clase, molecule, formulare și altele. Autentificarea și autorizarea utilizatorilor se realizează prin token-uri JWT, cu middleware dedicat.

Validarea datelor și gestionarea erorilor sunt tratate consistent, cu înregistrare de metrici pentru succes și eșecuri, ceea ce permite monitorizarea calității serviciilor.

Pentru interacțiunea cu MongoDB, structurile Go sunt mapate la documente BSON, cu chei explicite și suport pentru ID-uri generate automat. Structurile BSON sunt convertite nativ în JSON pentru serializarea/deserializarea datelor și trimiterea lor între servicii și front-end. Se poate observa un exemplu de generare și parsare a token-ului în structuri de date în anexa [A.1](#).

6.1.2 Implementarea serviciilor

cpp-compiler-service

Este serviciul responsabil pentru compilarea și executarea codului C++ transmis de către utilizatori în secțiunea de informatică. Acesta utilizează un server HTTP implementat în Go pentru a primi cereri de compilare și a trimite rezultatele înapoi către front-end. Serviciul primește un `CodeExecutionRequest` ce conține codul C++ al utilizatorului

și inputul necesar execuției, transformându-l pentru a include un sistem de urmărire a execuției.

După validarea cererii, serviciul creează un director temporar pe server în care sunt plasate fișierele necesare, inclusiv fișierele de urmărire, care permit trasarea pașilor executați în timpul rulării codului. Utilizând compilatorul g++, codul este compilat, iar eventualele erori sunt gestionate și trimise înapoi utilizatorului. În continuare, executabilul generat este rulat și rezultatele sunt colectate, inclusiv pașii de execuție detaliați, care sunt apoi transformați într-un format ușor de procesat pentru a fi prezentate în platformă.

Serviciul este construit pentru a gestiona atât erorile de compilare, cât și erorile de execuție, fiind capabil să returneze informații detaliate despre fiecare pas al execuției. De asemenea, timpul de execuție este limitat pentru a evita blocarea back-endului.

Cpp-compiler-service este implementat scalabil și poate fi extins pentru a include noi funcționalități, cum ar fi analiza codului pentru optimizări automate sau integrarea cu alte platforme externe.

evaluation-service

Serviciul gestionează întreaga funcționalitate legată de testele din aplicație, inclusiv crearea, actualizarea și obținerea de quizuri. Datele pentru fiecare quiz sunt stocate în MongoDB. Ele conțin un set de întrebări, fiecare întrebare având un text, opțiuni multiple de răspuns și indicarea răspunsului corect. Atunci când un student trimite răspunsurile pentru un quiz, acestea sunt validate, iar scorul final este calculat în funcție de corectitudinea răspunsurilor, aplicând eventual bonusuri pentru timp sau scor perfect.

Modelele de date, cum ar fi Quiz, Question și QuizResult, sunt definite folosind structuri în Go. De exemplu, pentru fiecare quiz, este creat un document Quiz care conține informații despre titlu, ID-ul clasei și al profesorului, întrebările asociate și rezultatele obținute de studenți. Datele sunt manipulate prin interogări și operații de modificare folosind driverul de MongoDB. Atunci când se creează un quiz, datele sunt validate pentru a se asigura că fiecare întrebare conține opțiuni și răspunsuri corecte. În cazul în care datele nu sunt valide, răspunsurile eronate sunt gestionate prin returnarea unui mesaj de eroare corespunzător.

Pe măsură ce studenții completează quizurile, rezultatele acestora sunt stocate în colecția de rezultate ale quiz-urilor, iar scorurile sunt actualizate în timp real. Fiecare rezultat al unui student este legat de un quiz specific, iar înregistrările sunt modifi-

cate folosind operații MongoDB de tip actualizare sau inserare. În plus, statisticile de performanță, precum scorul mediu, timpul mediu de completare și numărul de scoruri perfecte, sunt calculate periodic și actualizate în cadrul fiecărui quiz. Aceste statistici sunt esențiale pentru analiza performanței generale a quizurilor și pentru generarea de recompense pentru elevi. Datele sunt structurate într-un model flexibil care permite adăugarea de noi întrebări, actualizarea acestora și modificarea criteriilor de evaluare în mod eficient.

feed-data-service

Serviciul gestionează datele educaționale din platformă (formulele de la toate materiile), incluzând arii, volume, formule fizice, formulele chimice, statistici despre planetelor și elementele din tabelul periodic. Acesta interacționează cu frontend-ul, expunând endpoint-uri pentru manipularea și vizualizarea datelor, procesând fișierele mol și realizând conversii între diferite formate de date. Modelele utilizate în acest serviciu includ Molecule, Feed, Formula și PeriodicElement, care sunt stocate și gestionate în baza de date MongoDB.

Pentru gestionarea moleculelor, serviciul procesează fișierele mol încărcate, folosind funcții precum ParseMolFile din pachetul prettier dezvoltat, care extrage informații despre atomi și legături chimice. Aceste date sunt salvate într-un obiect Molecule, care conține atât informațiile brute (cum ar fi formula și descrierea), cât și datele prelucrate (atomii și legăturile). Partea de procesare include verificarea și completarea valorilor lipsă, validarea tipurilor de date și gestionarea erorilor în cazul fișierelor invalide. De asemenea, fiecare moleculă are un ID unic generat automat, iar toate modificările sunt stocate în colecțiile MongoDB.

În plus, feed-data-service se ocupă și cu gestionarea formulelor chimice și a elementelor din tabelul periodic, oferind funcționalități de creare, actualizare și ștergere a acestora. Datele elementelor, cum ar fi simbolul, numărul atomic, grupa și perioada, sunt stocate în structura PeriodicElement, iar pentru fiecare feed se asociază un set de formule chimice, care sunt actualizate și procesate pentru a asigura integrarea corectă cu celelalte componente ale platformei. În acest mod, serviciul asigură că toate datele educaționale sunt actualizate și disponibile pentru utilizatorii finali.

user-data-service

Serviciul de gestionare a datelor utilizatorilor se ocupă cu stocarea și procesarea tuturor datelor care sunt introduse sau pot fi extrase legate de utilizatorii platformei.

Aici apar structuri complexe ca User care stochează ID, email, hash parolă, rol, clase,

quiz_results, total_points, points_history, badges, ranguri. Badge-urile au ID, titlu, descriere, tip, iconiță, timestamp. UserBadge salvează progres, valoare curentă, grad de completare, timestamp.

Recompensele și badge-urile sunt gestionate de un controler separat ce expune metode ca EnsureBadgesExist care definește badge-uri implicite și le inserează dacă nu există. Altă metodă expusă este CheckAndUpdateBadges care preia user_quiz_history și badge-urile și calculează progresul, crează sau actualizează badge-urile utilizatorului curent.

Gestionare utilizatori și autentificarea se face prin endpoint-uri REST, cum ar fi Register, Login și GetMe. Register validează datele primite, face hash-ul parolei cu bcrypt, inserează user-ul în baza de date și incrementează metrica utilizatori activi. Login verifică hash-ul parolei și generează JWT care încapsulează rolul și email-ul. GetMe decodează token-ul, returnează datele profilului fără parolă desigur.

Fluxul de recompense și badge-uri este gestionat printr-un sistem de evenimente. La primirea rezultatelor din evaluation-service, handlerul decodifică câmpul QuizResult-Meta și salvează datele în baza de date users-data. Punctele calculate (base, time, streak, perfect) se adaugă în istoricul de punctaj și la punctele totale. UpdateUserRankings recalculează rangurile globale și pe clase.

6.1.3 Conectivitate

Microserviciile comunică prin API REST, iar traficul este expus și rulat în interiorul clusterului Kubernetes. Pentru dezvoltare locală, se utilizează port-forwarding Kubernetes pentru a expune temporar serviciile externe către mașina locală. Gestionarea rutelor externe și a împărțirii numărului de cereri se realizează cu ajutorul Kong Ingress Controller, care asigură rutarea traficului HTTP către serviciile potrivite.

În anexa [A.2](#) putem să vedem un exemplu de implementare a unui endpoint pentru crearea unui quiz, care primește datele de la front-end, le validează și le inserează în baza de date MongoDB, iar apoi răspunde cu codul HTTP corespunzător.

Rutele tipice sunt configurate în routerul fiecărui microserviciu, peste care se aplică middleware-ul JWT pentru protejarea endpoint-urilor sensibile. Accesul la endpointuri gestionat prin CORS este relaxat în mediul de dezvoltare pentru a permite testarea fără restricții de origine.

6.1.4 Procesarea codului în editorul interactiv

Pentru procesarea codului C++ introdus de utilizator în editorul interactiv, s-a implementat un serviciu dedicat care primește codul sursă, îl compilează și execută, returnând rezultatele intermediare în format JSON.

Pentru a vizualiza stările fiecărui container la runtime s-au parcurs următorii pași:

1. Se inserează antetul `tracer.h` și instanța globală a lui în fișierul C++.
2. Funcția `Go TransformCode` parcurge sursa linie cu linie și:
 - detectează declarații de containere STL (vector, map, stack, etc.);
 - generează pentru fiecare un `ContainerTracer` cu `makeTracedContainer()`;
 - inserează apeluri `traceOperation()` după operațiile semnificative
 - adaugă `getInstance().snapshot()` la return și după structuri de control.
3. La rulare, fiecare operație scoate la ieșirea standard linii prefixate cu `STATE:`.
4. Backend-ul Go parsează aceste linii, le convertește într-o listă de pași de execuție și le trimite frontend-ului.

```
1 #include "tracer.h"
2 Tracer* globalTracer = &Tracer::getInstance();
3 auto v_tracer = tracedContainer("v", v);
4 v.push_back(42);
5 v_tracer.traceOperation("insert", "Inserted_element");
```

Listing 6.3: Injectarea antetului și singleton-ului tracer

6.2 Front-end

Pentru dezvoltarea front-end-ului am folosit React, împreună cu React Router DOM¹ pentru navigare, tehnologia Three.js prin biblioteca React Three Fiber (R3F) și Drei pentru componente 3D reutilizabile. Această combinație a permis crearea unei interfețe interactive cu multiple vizualizări 3D dinamice.

¹<https://reactrouter.com/>

6.2.1 Configurare front-end

Mediul de dezvoltare a fost configurat cu Node.js¹ și npm², folosind Vite³ ca agregator și server de dezvoltare rapidă. În directorul de pachete ale proiectului, pachetele relevante instalate includ react, react-router-dom, react-three/fiber, react-three/drei și alte dependențe utile pentru animații și stilizare. Tehnologiile menționate se îmbină cu folosirea Tailwind CSS⁴ pentru stilizare rapidă și eficientă, permițând crearea de interfețe responsive.

6.2.2 Dezvoltare front-end

Componenta principală a aplicației este structurată pe baza React și React Router pentru navigare între pagini. Pentru vizualizările 3D s-au folosit componente R3F, cu suport Drei pentru facilități precum useGLTF pentru importul modelelor 3D, useSpring pentru animații și Canvas pentru spațiul 3D.

Exemple de componente dezvoltate în cadrul proiectului includ pagini de întâmpinare pentru materii, fiecare cu descrieri detaliate și elemente interactive. De asemenea, s-au dezvoltat formulare, precum ChemistryAddForm, care permit adăugare de molecule în listă prin încărcare de fișiere de descriere. S-au creat componente precum Chemistry-Balloon, Drone, Island și SpaceBackground, care sunt animate și controlate prin intrări de la utilizator. Alte componente includ elemente educaționale pentru structuri de date și concepte de programare, completate cu vizualizări grafice. În plus, s-au dezvoltat panouri de control și pagini pentru quiz-uri, clase, invitații și alte funcționalități, toate acestea având apeluri către API-ul backend-ului pentru a asigura consistență în datele afișate și posibilitatea de modificare a lor.

Un exemplu de componentă standard React care utilizează Three.js pentru a crea un balon animat se află la zona din anexe A.3. Această componentă folosește useSpring pentru a anima poziția și rotația balonului.

Alt exemplu este componenta meniului principal, care include o insulă complexă animată ce poate fi rotită cu mouse-ul în anexa A.4. Aceasta folosește useRef pentru a manipula direct obiectele Three.js și a aplica rotații în funcție de mișcarea mouse-ului.

În alte zone componentele sunt implementate după modelul de carduri, cum ar fi în

¹<https://nodejs.org/>

²<https://www.npmjs.com/>

³<https://vitejs.dev/>

⁴<https://tailwindcss.com/>

zona panoului de metrice bazate pe rezultatele obținute de elevi la diferitele quiz-uri. Se folosesc și componente deja existente precum containere responsive și adaptive la dimensiunile ecranului. Unele componente fac și validări de date sau agregări scurte pe datele pe care le primesc, procesările și agregările importante fiind realizate în backend.

Majoritatea componentelor urmează o structură clară, întâi în definiția componentei fiind state-urile și funcțiile de manipulare a acestora, apoi hook-urile React pentru efecte secundare, apoi filtre sau efecte care nu sunt principale și în final returnarea JSX-ului care definește interfața grafică.

Unele scene 3D încep cu interfețe de declarare de elemente, precum geometria unor obiecte, materiale sau chiar și animații. Componentele au logica de animare în general în `useFrame`, iar state-urile sunt legate cu `useRef` sau `useState` pentru a permite actualizarea lor în timp real. După asta apar funcțiile de procesare sau utilitare, iar componentele se termină cu returnarea JSX-ului care definește structura vizuală a scenei ca și în celelalte cazuri.

6.2.3 Implementare front-end

În partea de front-end, aplicația este structurată pe componente React, folosind TypeScript în unele zone pentru tipare strictă și TailwindCSS pentru stilizare. Fiecare componentă 3D (de exemplu `BadgeDisplay`, `BalloonMascot`, `Balloon` sau `Island`) este definită printr-un set de interfețe care descriu proprietățile primite (cum ar fi `BadgeProps` sau `BalloonProps`). Aceste componente folosesc hook-uri de stare (`useState`) și referințe (`useRef`) pentru a păstra stările de atingere a secțiunii, selecție și animații, iar `useFrame` asigură actualizarea fiecărui cadru direct pe obiecte (rotire, scalare, etc).

Animațiile neon și efectele de glow sunt obținute cu `meshPhongMaterial` și proprietăți dinamice de `emissiveIntensity` și `opacity`, corelate cu starea de atingere a secțiunii. Pentru mișcarea obiectelor 3D se folosește componenta `Float` din `rei`, iar pentru text 3D componenta `Text` din `rei`. Camera este controlată în scene printr-o componentă custom (`CameraController`) care implementează rotire și mișcare inerțială, iar interacțiunea mouse-ului e centralizată în `SceneController` prin gestionarea evenimentelor la mișcare.

Pentru formulare și interfețe modulare (de ex. `ChemistryAddForm`, `ListFormulas`, `ClassPerformanceCard`) s-a folosit React clasic cu `useEffect` pentru a accesa API-ul, `useState` pentru gestionarea datelor (`loading`, `error`, `succes`) și componente de UI animate cu `Framer Motion`. În `ChemistryAddForm`, încărcarea fișierelor `.mol` este realizată cu un `FileReader` în `handleFileChange`. Toate cererile includ antetul JWT extras din stocarea locală.

Pentru grafice de date statistice s-a integrat `recharts`, configurând `BarCharts`, `ResponsiveContainers` și `CartesianGrids`, cu transformări simple ale datelor în codul din `useEffect`. Distribuția principală (sidebar, navbar) folosește componente UI din biblioteca `lucide-react` pentru iconițe și `React Router` pentru navigare, împreună cu `Tailwind` pentru starea activă/inactivă și efecte vizuale.

6.2.4 Conectivitate

Aplicația comunică cu backend-ul prin intermediul `fetch API`, efectuând cereri REST către serverul local pe portul 8000, în contextul mediului de dezvoltare utilizat. De exemplu, pentru obținerea datelor precum formulele, moleculele sau quiz-urile, aplicația utilizează `useEffect` în componente pentru a asigura actualizarea datelor instant. În ceea ce privește autentificarea, token-ul JWT este gestionat și salvat în stocarea locală, fiind folosit ulterior în antetul `Authorization` al cererilor din API.

De asemenea, s-a implementat un sistem de tratare a erorilor, care afișează mesaje relevante utilizatorului atunci când apar probleme în comunicarea cu serverul. În plus, au fost dezvoltate formulare pentru inserarea, ștergerea și actualizarea datelor pe server, iar răspunsurile și validările sunt gestionate vizual în frontend pentru o experiență mai bună.

Se folosește `useEffect` pentru a chema callback-urile de JavaScript care iau datele de la backend și le afișează în interfață. Acest lucru permite ca datele să fie actualizate în mod dinamic, fără a fi nevoie de reîncărcarea paginii.

Iar o secțiune importantă este vizualizarea 3D a moleculelor, care primește datele de la backend și crează o reprezentare 3D a atomilor și legăturilor chimice folosind `Three.js`. Se folosește de asemenea `useEffect` pentru a încărca datele și apoi le poziționează într-un grup de obiecte `Three.js`. Acest lucru se poate observa în anexa [A.5](#).

Crearea unei scene dinamice ca cele din secțiunea de informatică pentru structuri de date se face prin simularea lor manual pentru a vedea rezultatul operațiilor și conținutul lor în fiecare moment. Apoi se crează o scenă 3D demo care afișează nodurile și conexiunile lor folosind `Three.js` și `React Three Fiber`. Diferitele structuri de date sunt construite sub diverse forme sugestive pentru particularitățile lor. De exemplu, cele care păstrează ordinea elementelor sunt afișate ca arbore de căutare cu auto echilibrare deoarece așa sunt implementate în biblioteca standard din C++. Acestea sunt reprezentate cu noduri și conexiuni între ele care se ajustează automat în funcție de operațiile efectuate. Alte structuri sunt afișate liniar dacă e cazul sau cu forme specifice ca bucket-urile pentru hashtable-uri sau stivă și coadă cu modelele lor consacrate.

Capitolul 7

Prezentarea funcționalităților finale ale aplicației

7.1 Înregistrare și autentificare utilizator

Înregistrarea și autentificarea utilizatorilor se face după un proces clasic, în care dacă utilizatorul există deja, se va autentifica cu numele de utilizator și parola, iar dacă nu există, se va înregistra cu un email și o parolă. În caz de succes la înregistrare se trimit datele în back-end, se stochează în baza de date, se va afișa un mesaj de succes și se va porni animația de succes la înregistrare a mascotei balon. După înregistrare, utilizatorul poate intra în pagina de logare și se poate autentifica cu datele introduse la înregistrare. Înregistrarea are două moduri posibile pentru conturi de profesor și conturi de elev.

În cazul în care utilizatorul nu există, se va afișa un mesaj de eroare și se va rămâne pe pagina de logare. În cazul în care credențialele sunt corecte și există în sistem se va afișa un mesaj de succes, se va face animația mascotei balon pentru logare și se va redirecționa utilizatorul către pagina principală a aplicației. Modelul 3D al mascotei este responsiv și la scrierea caracterelor în promptul de înregistrare și logare.

7.2 Explorarea meniul principal

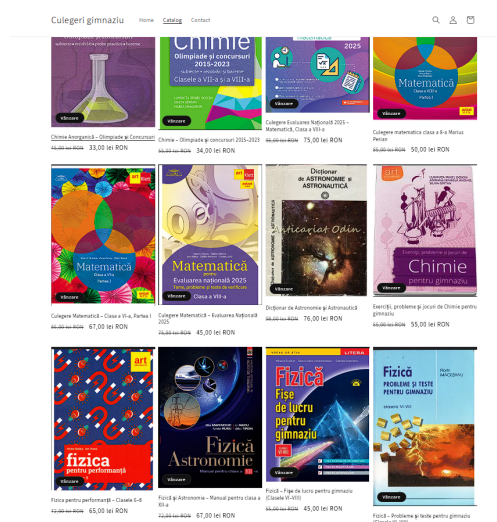
Meniul principal al aplicației este afișat după autentificare și conține un model 3D principal cu o insulă mare și mai multe insule mici care reprezintă secțiunile educaționale disponibile în aplicație. Navigarea se face prin rotire cu mouse-ul. Restul meniului este reprezentat printr-o bară clasică de navigare în partea de sus a ecranului, care conține

butoane pentru accesarea profilului, magazinului de cărți, secțiunilor despre și contact.

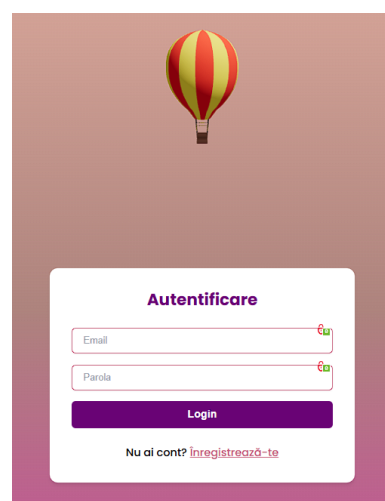


Figura 7.1: Interfața meniului principal

Magazinul online este o secțiune dedicată unde utilizatorii pot achiziționa cărți educaționale care a fost realizat separat de aplicație utilizând tehnologia Shopify¹ și care este disponibil de lansare către producție împreună cu platforma finală. Aceasta poate reprezenta și o sursă de venit afiliat platformei care este gratuită. Pagina de contact este o secțiune unde utilizatorii pot trimite feedback, sugestii sau întrebări către dezvoltatorii platformei.



(a) Magazinul online



(b) Pagina de login

Figura 7.2: Magazinul online și pagina de login

¹<https://www.shopify.com/>

7.3 Accesarea secțiunilor educaționale

Fiecare secțiune conține multiple scene 3D educaționale care reprezintă noțiuni sau fenomene specifice domeniului respectiv. Cu fiecare dintre ele se poate interacționa prin rotirea camerei, zoom și mișcarea camerei în jurul scenei. Pentru secțiunea de geometrie avem următoarele scene: Cub, Piramidă, Paralelipiped, Cilindru, Con și Sferă.

Pentru materia de fizică avem scenele: plan înclinat, scripete simplu, scripete mobil, pendul și oscilații, resort și legea lui Hooke, mișcarea circulară uniformă, mișcarea de proiectil, cădere liberă, coliziune elastică și coliziune plastică.

Pentru materia de chimie avem scene educaționale personalizabile prin încărcarea manuală a moleculelor chimice în format .mol, ele fiind parsate și transmise înapoi la front-end care le transformă în scene 3D.

În cazul scenelor de astronomie avem suport pentru toate planetele din sistemul Solar, cât și pentru sistemul Solar în sine cu animația rotațiilor planetelor în jurul Soarelui.

Ultima materie suportată curent este informatica care conține scene educaționale despre structuri de date și algoritmi, cum ar fi: array, vector, unordered map, ordered map, unordered set, ordered set, multiset, priority queue, deque, stack, queue, list și doubly linked list.

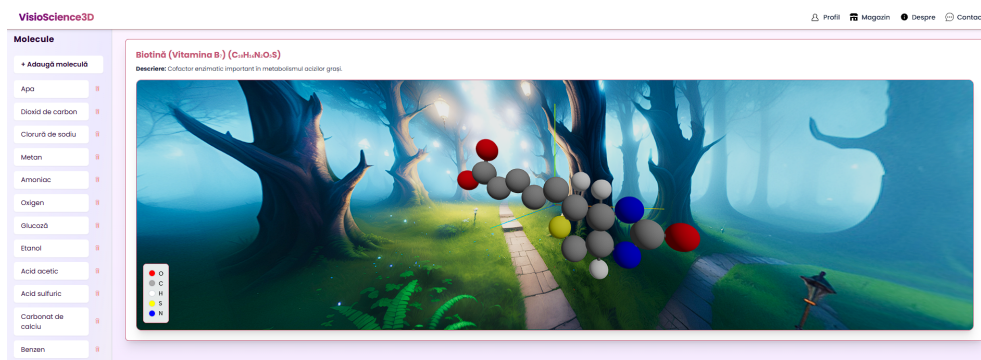


Figura 7.3: Exemplu de scenă de chimie

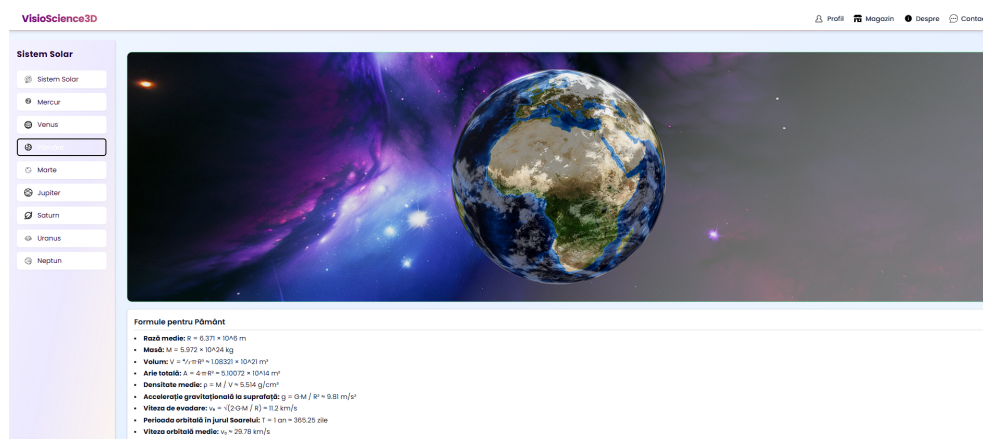


Figura 7.4: Exemplu de scenă de astronomie

7.4 Interacțiunea cu scenele 3D educaționale

Pe lângă interacțiunea clasică cu scenele 3D prin rotirea camerei, zoom și mișcare, la secțiunea de chimie se pot defini propriile molecule chimice prin fișiere cu format .mol. Alt tip de interacțiune este de exemplu prin operațiile de modificare a structurilor de date și algoritmilor din secțiunea de informatică, unde se pot adăuga sau elimina elemente din structurile de date în cauză, se pot vizualiza elementele și se pot anima operațiile de inserare, ștergere sau căutare a elementelor.

(a) Formular de încărcare molecule

Deque <T> Demo

(b) Operații pe structuri de date

Figura 7.5: Interacțiunea cu scenele 3D educaționale

7.5 Gestionarea profilului de profesor

Profesorii au panouri complexe de control asupra resurselor și contului lor. În secțiunea ce urmează se vor prezenta funcționalitățile principale disponibile.

7.5.1 Crearea de clase și gestionarea elevilor

Profesorul are acces la crearea de clase și gestionarea elevilor din aceste clase prin panourile de administrare. Acesta poate adăuga elevi noi în clasele sale, poate vizualiza lista de elevi și poate edita informațiile acestora.

7.5.2 Crearea de teste și gestionarea lor

Profesorul are de asemenea acces la crearea de teste pentru elevii săi. Aceste teste pot fi personalizate cu întrebări și răspunsuri, simple sau cu răspuns multiplu, cu sistem de punctaj customizabil.

7.5.3 Vizualizarea rezultatelor

Profesorul poate vizualiza rezultatele testelor elevilor săi, inclusiv scorurile și răspunsurile la întrebări. Profesorii au și un panou complex integrat pentru a vizualiza metrice mai complexe legat de rezultatele elevilor, despre care se va discuta ulterior.

7.6 Gestionarea contului de elev

7.6.1 Intrarea în clasele profesorului

Intrarea într-o anumită clasă se face pe bază de invitație de la profesor, direct din platformă în panoul de control al elevului. Elevul poate vedea clasele la care este înscris și poate accesa resursele și testele disponibile în acele clase.

7.6.2 Accesarea testelor și vizualizarea rezultatelor

Testele au interfață proprie de vizualizare și completare, unde elevul poate răspunde la întrebări și poate vedea scorul obținut după finalizarea testului.



Figura 7.6: Interacțiunea elevului cu testele

7.6.3 Compilatorul interactiv de cod

La zona de informatică avem și suport pentru compilatorul interactiv descris anterior. Poate fi folosit pentru a vedea în timp real rularea codului prin pașii prin care trece fiecare structură de date sau algoritm.

Se poate observa cum arată partea vizuală din editorul de cod. În partea stângă avem pașii de execuție extrași din cod după rulare și expuși sub formă de listă prin care se trece prin butoanele de navigare sau prin cel de rulare automată. În partea dreaptă avem intrarea pe care o dăm codului și ieșirea pe care o generează codul după rulare. În partea de jos avem scena 3D cu structurile de date identificate care se actualizează în timp real cu fiecare pas de rulare a codului prin care trecem cum se poate observa mai jos.

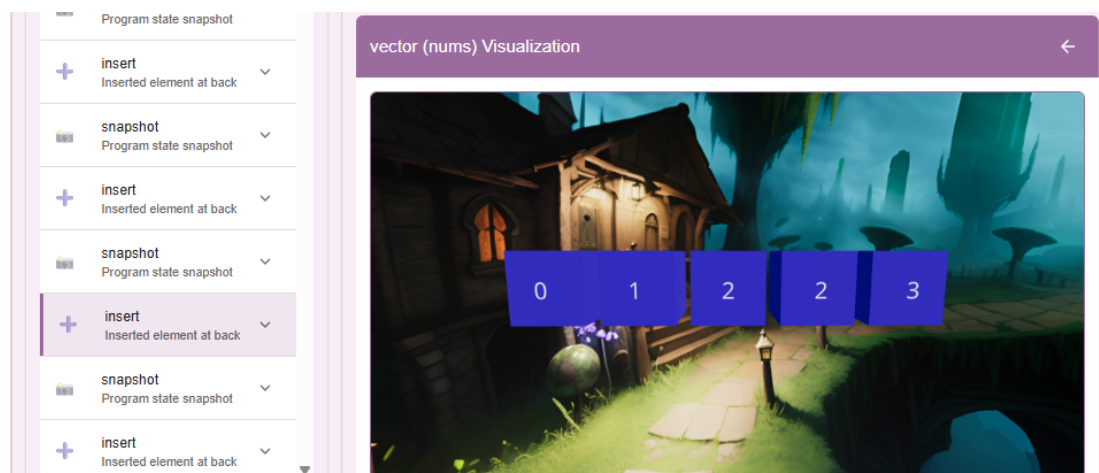


Figura 7.7: Editorul vizual cu pași de rulare

Această zonă este integrată în pagina de editare a codului împreună cu partea unde se scrie codul în sine. În partea dreaptă se află ce am descris anterior. În partea stângă avem editorul care folosește tema de Visual Studio Code¹ și care are suport pentru limbajul C++ la momentul actual. El suportă redimensionare, copiere automată a codului sau resetare la exemplul de cod inițial.

7.6.4 Tabelul de elemente interactiv

La zona de chimie avem și un tabel periodic Mendeleev interactiv care permite elevilor să interacționeze cu elementele chimice și să le dispună în diferite forme. Tabelul este responsiv și reactiv la selectarea oricărui element în orice poziție și deschide un panou de detalii despre acel element care arată numărul atomic, simbolul chimic, masa atomică, grupa, perioada și o descriere pe scurt despre elementul selectat.



Figura 7.8: Tabloul de elemente chimice interactiv

Elementele au multiple moduri de așezare interactivă, precum tabel, sferă, helix, grid, spirală, piramidă, val, floare, torus, cub, galaxie, ADN, copac, vortex, stea, fagure, solar, fractal sau matrice.

7.6.5 Panoul profesorului

Profesorul are acces la diferite metrice actualizate automat despre elevi, clase, teste și resurse. Există patru secțiuni principale în panoul profesorului. Prima este clasamentul elevilor, unde se poate vedea topul elevilor după rezultatele obținute la teste. A doua este secțiunea de performanță a clasei unde se poate vedea scorul mediu în clasă,

¹<https://code.visualstudio.com/>

procentul de îmbunătățire a clasei și performanța fiecărui test per clasă. A treia este secțiunea de îmbunătățiri ale elevilor, unde vedem detalii ca performanța generală, tendințe de performanță și elevii cu cea mai mare îmbunătățire. Ultima secțiune este cea de statistici per test, unde se pot vedea detalii despre rata de completare, distribuția dificultății, scorurile medii per test și statistici detaliate despre fiecare test în parte, precum rata de greșeli, totalul de încercări, răspunsurile corecte și cele greșite.

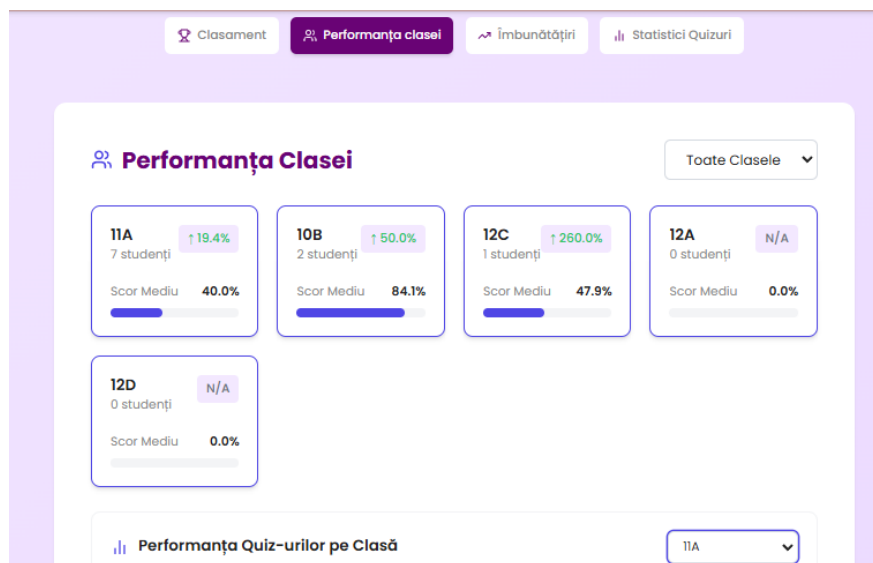


Figura 7.9: Panoul profesorului cu metrice și statistici

7.6.6 Recompensele elevilor

Elevii primesc insigne 3D sub formă de baloane tematice ca și mascota platformei pentru realizările lor. Acestea sunt afișate în panoul de recompense al elevului și pot fi obținute pentru diverse realizări, precum completarea primului test, obținerea unui scor maxim la un test, ajungerea la rang de expert prin rezolvarea unui număr mare de teste, etc.

Capitolul 8

Evaluarea implementării

8.1 Introducere

Obiectivele stabilite pentru platformă au fost atinse, iar evaluarea implementării arată un succes în ceea ce privește funcționalitatea, performanța și experiența utilizatorului.

8.2 Răspunsurile de posibili utilizatori

În urma formularului discutat și la secțiunea motivație, au fost incluse și întrebări care dau o notă de la 1 la 5 pentru fiecare tip de simulare 3D de la diferite materii din platformă în videoclipuri atașate. Au fost două tipuri de întrebări, una care cerea să se ofere o notă interactivității și una care cerea să se ofere o notă utilității simulărilor. S-a înregistrat un număr de peste 35 de răspunsuri.

Pentru simulările de fizică peste 71% dintre respondenți au dat nota maximă 5 pentru interactivitate și peste 68% au dat nota maximă 5 pentru utilitate. Pentru exemplele de scene de chimie din videoclipurile atașate, peste 64% dintre respondenți au dat nota maximă 5 pentru interactivitate și peste 68% au dat nota maximă 5 pentru utilitate. Pentru simulările din zona de informatică, peste 71% dintre respondenți au dat nota maximă 5 pentru interactivitate și peste 65% au dat nota maximă 5 pentru utilitate. De menționat este și că la toate întrebările peste 95% dintre respondenți au dat o notă mai mare sau egală cu 3/5 pentru interactivitate și utilitate la toate materiile incluse.

În final, ca evaluare generală a platformei, am întrebat și dacă ar fi fost utilă o platformă ca aceasta dacă respondenții ar fi avut acces la ea în școală. Rezultatele se pot vedea mai jos, cu peste 90% dintre răspunsuri indicând că sunt de acord sau foarte de acord

că o platformă ca aceasta ar fi fost utilă.

Dacă aş fi avut o platformă ca aceasta în liceu/gimnaziu, mi-ar fi îmbunătăţit înţelegerea conceptelor la diferite materii din domeniul ştiinţific?

36 responses

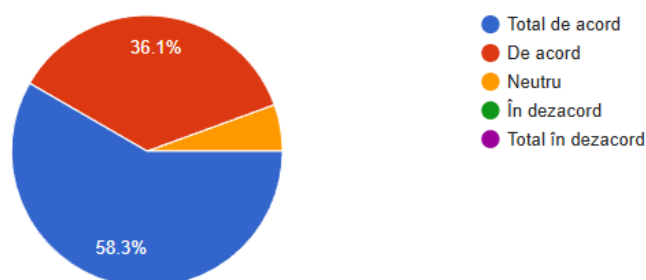


Figura 8.1: Ar fi fost utilă o platformă ca aceasta în şcoală?

8.3 Evaluarea back-endului

8.3.1 Testarea back-endului

Testarea back-endului s-a concentrat pe validarea funcţionalităţilor API-urilor dezvoltate pentru interacţiunea cu baza de date şi manipularea datelor. Pentru aceasta, au fost utilizate soluţii precum Postman¹, curl² şi scripturi locale pentru a trimite cereri. În plus, au fost testate diverse scenarii de utilizare pentru a verifica integritatea fluxului de date între componentele backend-ului şi frontend. Acestea includ inserarea, modificarea şi ştergerea de date în MongoDB, precum şi verificarea corectitudinii răspunsurilor API-urilor.

8.3.2 Monitorizarea back-endului

Pentru monitorizarea back-endului, s-au utilizat instrumente de adăugare şi colectare precum Prometheus şi Grafana pentru a urmări performanţa sistemului şi a verifica orice problemă de scalabilitate sau fiabilitate. Datele colectate au arătat o utilizare constantă de resurse, cu peste 95% din cereri servite în mod eficient, fără a depăşi limitele impuse de infrastructura curentă.

¹<https://www.postman.com/>

²<https://curl.se/>

8.3.3 Evaluarea performanței back-endului

Performanța back-endului a fost evaluată pe baza unui set de testare a scenariilor de utilizare normale. Testele au arătat o latență medie de răspuns de 250ms pe cereri simple și sub 2 secunde pentru încărcarea obiectelor mari din scene. În ceea ce privește scalabilitatea, s-au făcut teste de încărcare pentru a evalua comportamentul sistemului la un număr mare de cereri simultane, iar infrastructura a rămas stabilă, fără scăderi semnificative ale performanței chiar și testat în mediu de dezvoltare cu resurse locale limitate.

Alt nivel de testare în timpul dezvoltării a fost prin teste unitare, rulate local cu go test. Folosind pachetul httptest am simulat cereri HTTP către ruterul unui serviciu din interiorul podului și am putut itera rapid și prinde bug-urile, fără să reconstruim imaginea Docker sau să dăm deploy în Kubernetes de fiecare dată.

Rularea comenzii `go test ./...` rulează orice test rapid, oferind feedback aproape instant asupra regresiiilor de exemplu din transformatorul de cod, parser sau din logica de gestionare și propagare a erorilor de compilare către frontend.

8.4 Evaluarea front-endului

8.4.1 Testarea front-endului

Testarea front-endului a fost realizată prin testare manuală a interacțiunilor cu utilizatorul. S-au testat funcționalitățile de navigare, interactivitatea componentelor (de exemplu, meniurile rotative, butoanele de creare a quiz-urilor), și integrarea corectă a modelelor în scene. Testele au arătat că 95% dintre interacțiuni au fost realizate fără erori, iar platforma a funcționat decent pe mașina de testare.

8.4.2 Monitorizarea front-endului

Majoritatea paginilor se încarcă complet în mai puțin de 2 secunde pe majoritatea dispozitivelor, iar utilizarea memoriei este sub 200MB pentru majoritatea scenelor 3D care încarcă obiecte și texturi.

8.4.3 Evaluarea performanței front-endului

Performanța front-endului a fost evaluată prin testarea mai multor scenarii de utilizare, inclusiv interacțiuni cu animații 3D complexe și manipularea lor. Testele au arătat că

platforma susține animații 3D simultane cu un cadru constant de 60fps, iar timpii de încărcare au fost sub 1,5 secunde pentru majoritatea paginilor.

8.5 Testarea infrastructurii / platformei

S-au utilizat teste de scalabilitate, iar platforma a demonstrat un comportament robust și o scalabilitate bună în fața unui număr mare de cereri. De exemplu, un scenariu de testare a implicat multiple cereri simultane, iar sistemul a rămas stabil, fără erori sau timp de nefuncționare. În plus, în caz că apar probleme de performanță, se pot scala serviciile în mod dinamic, adăugând replici direct din scalarea oferită de Kubernetes pentru a răspunde la cerințele crescute.

8.6 Probleme întâmpinate

8.6.1 Backend & Kubernetes Ingress

Una dintre principalele provocări întâmpinate a fost accesibilitatea Kong Ingress Controller în mediul local, care nu este disponibilă implicit. Soluția implementată a fost folosirea unui port-forward pentru a expune Ingress-ul. Totodată, expunerea micro-serviciilor prin Kong a necesitat sincronizarea corectă a valorilor pentru containerPort și targetPort, configurarea serviciilor de tip ClusterIP și adnotările specifice pentru Ingress. Ajustările făcute au asigurat că traficul extern poate accesa serviciile într-un mod stabil și mai ales controlat.

8.6.2 Accesibilitate CI/CD Build & Rollout Workflow

Fluxul de lucru pentru CI/CD a fost un alt punct dificil de gestionat. Fiecare schimbare în cod presupunea reconstruirea și încărcarea imaginii containerului, urmată de un proces de rollout. Pentru a automatiza procesul folosind GitHub Actions astfel încât workflowul să aibă acces la clusterul local a fost nevoie de crearea unui GitHub runner local care să ruleze pe mașina de dezvoltare.

8.6.3 Epuizarea memoriei GPU din cauza texturilor masive

Un alt obstacol major a fost legat de modelele 3D ale Pământului și hărțile de fundal, care aveau rezoluții foarte mari (16K×8K). Acestea produceau erori de memorie în WebGL (pierdere de context, erori la shadere și avertismente de redimensionare a

texturilor). Problema a fost rezolvată prin utilizarea unor variante de 4K–8K pentru texturi, încărcare leneșă a acestora și folosirea formatelor de texturi comprimate pentru a reduce consumul de memorie și a preveni pierderile de context WebGL.

8.6.4 Crearea fluxului de parsare a codului C++

Cea mai dificilă parte a fost design-ul antetului `tracer.h`: au trebuit definite meta-trăsături care să distingă între vector, list, map, priority_queue sau chiar `std::array` și altele, fără ajutorul reflecției native asupra tipului. În plus, a fost nevoie de serializarea stării fiecărui container. Asta a complicat puternic manipularea tipurilor compuse, precum `std::pair<K,V>` din map-uri. Implementarea funcției auxiliare `serializePair()` și a ramurilor specializate din `getState()` / `getMetadata()` a solicitat multă muncă pentru a gestiona corect conversiile către JSON. În final s-a putut evita ambiguitatea dintre clasele STL și tipurile primitive. Nu în ultimul rând, menținerea consistenței între operațiile detectate în `TransformCode()` și cele raportate de template-urile `tracer` a implicat testare repetată, deoarece o singură pierdere genera stări incoerente la final.

8.6.5 Dimensiunea clusterului Kubernetes

Un alt obstacol a fost dimensiunea clusterului Kubernetes care este practic stocat prin imagini și volume Docker. Fără curățare constantă s-au ajuns la dimensiuni de aproape 60GB, ceea ce a dus la probleme de spațiu pe disc și la dificultăți în gestionarea resurselor. Soluția a fost curățarea imaginilor nefolosite și compresarea volumelor Docker, iar apoi compresarea întregii imagini de stocare `wsl`¹ de mai multe ori pentru a putea continua dezvoltarea.

8.6.6 Popularea și accesul extern la intrările în baza de date

Pentru a crea entități noi sau a adauga date educaționale în MongoDB într-un pod de Kubernetes, a fost necesară crearea unui job² dedicat care să aibă acces în interiorul clusterului, deoarece nu se poate accesa direct din afară cu cereri directe. Asta a însemnat crearea unui fișier de configurare YAML pentru jobul respectiv și crearea unui pod dedicat pentru rularea acestuia.

¹<https://learn.microsoft.com/en-us/windows/wsl/>

²<https://kubernetes.io/docs/concepts/workloads/controllers/job/>

8.7 Analiza metricilor

Cei mai reprezentativi indicatori de exploatare în contextul unei platforme ca VisioScience3D sunt timpii de răspuns, rata de cadre pe secundă (FPS) în scenele 3D, costurile de licențiere și consumul de memorie RAM. Timpul de răspuns a fost constant sub 250ms pentru majoritatea cererilor în mediul de dezvoltare, putând să difere în eventuala producție în funcție de încărcarea și performanțele serverului. Rata de cadre pe secundă este asigurată de tehnologiile WebGL și Three.js folosite și de tehnicile de optimizare incluse. În general se menține constantă la cel puțin 60 cadre pe secundă. Costurile de licențiere sunt zero pentru VisioScience3D, în timp ce alte platforme competitive implică taxe anuale semnificative. În ceea ce privește consumul de memorie RAM, platforma se menține sub 200MB în majoritatea scenelor, ne încărcând resurse de foarte mari dimensiuni.

Metricile au fost obținute prin utilizarea browserului Chrome pentru accesarea siteurilor și folosind informațiile de latență la nivel de cerere, resursele utilizate raportate de browser și rata de cadre resimțită în scenele 3D la utilizare (sub 60 de cadre pe secundă se resimt mici sacadări în animații și interacțiuni).

VisioScience3D răspunde în medie mai repede cel puțin în mediul de dezvoltare decât platforme gratuite și depășește nivelul minim de 60 cadre pe secundă, prag important pentru o experiență vizuală fluidă. În testele executate pe arhitectura backend-ului, aplicația a susținut până multiple cereri concurente fără să rescalaze pod-urile folosite. Din punct de vedere monetar, lipsa taxei de licențiere o poziționează peste Mozaweb și IQBoard prezentate la secțiunea Studiu de piață, care introduc un cost de utilizare semnificativ.

Capitolul 9

Concluzii și perspective

9.1 Concluzii

În concluzie, proiectul VisioScience3D reprezintă o soluție completă pentru gestionarea și monitorizarea performanțelor elevilor la materiile de științe sau în termeni internaționali STEM. Platforma integrează funcționalități robuste care asigură o experiență interactivă și captivantă utilizatorilor, demonstrând și un interes crescut prin studiul efectuat prin formularul distribuit. Soluția expune o interfață interactivă și un back-end scalabil după cum am descris și oferă astfel o soluție de viitor pentru nevoile elevilor și profesorilor.

Testele realizate au demonstrat că soluția implementată îndeplinește obiectivele stabilite, iar performanța și stabilitatea soluției trec evaluările de integrare finale făcute pe majoritatea scenariilor cu utilizatori reali, iar feedback-ul per total dat de testeri a fost pozitiv în ceea ce privește interactivitatea și utilitatea conceptului.

9.2 Dezvoltare viitoare

Pentru a îmbunătăți în viitor aplicația și a răspunde unor nevoi mai complexe, există mai multe direcții posibile de dezvoltare viitoare:

Funcționalități de export și analize suplimentare pe care le pot face profesorii:

Ar putea fi adăugat un buton de export a rezultatelor din leaderboard-uri și de la teste în general. S-ar putea adăuga suport pentru exportul datelor folosind formate precum CSV, PDF sau altele la cerere.

Integrarea de notificări și alerte: Implementarea unor notificări prin email pentru utilizatori ar putea îmbunătăți experiența acestora, anunțând despre teste viitoare, termene limită sau evaluări de performanță.

Integrarea cu modele populare de inteligență artificială (precum modele lingvistice de exemplu): Implementarea unor funcționalități de suport pentru elevi bazat pe inteligență artificială ar putea sprijini foarte mult procesul de învățare, oferind explicații detaliate și sugestii personalizate pentru întrebările elevilor de exemplu.

Funcționalități de colaborare: Oferirea unui sistem de chat în timp real integrat în aplicație ar putea facilita colaborarea între utilizatori, eliminând necesitatea utilizării unor alte platforme externe pentru comunicare.

Prin urmare, viitorul aplicației se axează pe extinderea funcționalităților, îmbunătățirea performanței și scalabilității și integrarea de noi tehnologii pentru a oferi utilizatorilor o experiență din ce în ce mai personalizată și interactivă.

Anexa A

Anexe

```
1 func GenerateToken(userID, role, email)
2   claims := CustomClaims(
3     userID, role, email, issuedClaims, until, issueDate)
4   return jwt.NewWithClaims(signingMethod, claims)
5 func ParseToken(tokenStr)
6   t, err := ParseWithClaims(tokenStr, customClaims,
7     if t.Method !valid return err else return jwtSecret)
8   if token valid return token else return token invalid
```

Listing A.1: Generare și validare token JWT

```
1 func CreateQuiz(writer, req)
2   var i QuizInput
3   if Decode(req.Body, &input) fails return err
4   classID, err := ToObjectID(input.ClassID)
5   if err != nil return Error(w, "Id_de_clasa_invalid")
6   quiz := {newObjId, i.Title, classID, i.Q&As, time}
7   if InsertOne("quizzes", quiz) fails return error
8   Respond(w, quiz, Created, 200)
```

Listing A.2: Funcția CreateQuiz pentru crearea unui quiz nou în MongoDB

```
1 const Baloon = ({ scale }) =>
2   const [state s, setState] = useState("idle");
3   useEffect => { interval = setInterval(() => setState(
4     ["idle", "left", "right", ..]
```

```

5      [Math.floor(random() * 7)], 2000);
6      return () => clearInterval(interval);}, []
7      const { position, rotation } = useSpring
8      position: s === "up" ? [..] : s === "down" ? [..] : [..],
9      rotation: s === "l" ? [..] : s === "r" ? [..] : [..],
10     config: { duration: 2000 }
11     return <mesh pos rotation scale>...</mesh>;

```

Listing A.3: Exemplu de componentă React pentru un balon animat

```

1  const Island = ({ isRot, setIsRot }) =>
2    const ref = useRef(), lastX = useRef(0), alfa = 0.0075;
3    const onDown = e => setIsRot(true); lastX.curr = e.clientX;
4    const onUp = () => setIsRot(false); ...
5    useEffect(() => {
6      window.addEventListener("down", onDown);
7      ... return () => { ... }; }, [rotating]);
8    return <a.group ref={ref}>...</a.group>;

```

Listing A.4: Exemplu de animație a insulei meniului principal

```

1  const Molecule = ({moleculeId}) =>
2    const [atoms,setAtoms] = useState([]);
3    const [bonds,setBonds] = useState([]);
4    useEffect() => fetch => r.json() => (setAtoms, setBonds)
5    useEffect() => if(groupRef.current)
6      const box = new THREE.Box3().setFromObj(groupRef.curr)
7      groupRef.current.position.sub(box.getCenter(center))
8    return <group ref={groupRef}> atoms.map((a,i)=><mesh key pos
9      ><sphere args={} /><mesh color />)
10      bonds.map(b,i)=> const start=atoms[b.a1], end=atoms[b.a2];
11      if(!start||!end) return null;
12      const startV=new THREE.Vector3(start.x,start.y,start.z);
13      const endV= new THREE.Vector3(end.x,end.y,end.z);
14      const mid= startV.clone().add(endV).multiply(0.5);
15      const dir= endV.clone().sub(startV).normalize();
16      const len= startV.distanceTo(endV);
17      const quat= Quaternion().setFrom(Vector3(0,1,0), dir);
18      return <mesh key=i pos=mid quaternion=quat><cylinder
19        args={} /><standardMaterial color={} /></mesh>;

```

Listing A.5: Vizualizare 3D Molecule

Bibliografie

- [1] Milada Teplá, Pavel Teplý, “Influence of 3D models and animations on students in natural subjects” International Journal of STEM Education, 2022. <https://stemeducationjournal.springeropen.com/articles/10.1186/s40594-022-00382-8> [Accesat 18 mai 2025].
- [2] Gina-Nuța Isofache, “Distribuția stilurilor de învățare în rândul elevilor din România,” Revista Profesorului, 2019. <https://revistaprofesorului.ro/studiu-privind-invatarea-vizuala/>. [Accesat 20 mai 2025].
- [3] “Stilurile de învățare,” ITeach. <https://iteach.ro/pagina/1113/>. [Accesat 21 mai 2025].
- [4] Ting Li, Yidan Hu, Ruohong Song, Hangyu Liu, Xin Gao, Fei Wang, Chengwei Xiao, Xilin Liu, “Unveiling the ultimate advantage: a meta-analysis of 3D visualization and problem-based learning in orthopedic education”, PubMed, 2025. <https://pubmed.ncbi.nlm.nih.gov/39806348/>. [Accesat 22 mai 2025].
- [5] Frontiers in Education, “Adding immersive virtual reality laboratory simulations to traditional teaching methods enhances biotechnology learning outcomes”, 2024. frontiersin.org/feduc.2024.1354526/full. [Accesat: 23 mai 2025].
- [6] Mindomo, “What is Visual Learning, and How can it Help Retain and Comprehend Information Easier?”, 2022. <https://www.mindomo.com/blog/what-is-visual-learning/>. [Accesat 24 mai 2025].
- [7] OECD Education Today, “Can the Targeted Use of Digital Devices win over the naysayers?” 2025. <https://oecdeditoday.com/can-the-targeted-use-of-digital-devices-in-education-win-over-the-naysayers/>. [Accesat 26 mai 2025].
- [8] Haozhe Jiang, Deqi Zhu, Ritesh Chugh, “Virtual reality and augmented reality-supported K-12 STEM learning: trends, advantages and challenges” Springer

- Nature Link, 2025. <https://link.springer.com/article/10.1007/s10639-024-13210-z>. [Accesat 28 mai 2025].
- [9] ResearchGate, “Visual Learning: The Power of Visual Aids and Multimedia,” 2024. https://www.researchgate.net/publication/385662029_Visual_Learning_The_Power_of_Visual_Aids_and_Multimedia. [Accesat 29 mai 2025].
- [10] ResearchGate, “Teacher Satisfaction and Their Perception of Teaching Platforms,” 2024. https://www.researchgate.net/figure/Teacher-satisfaction-and-their-perception-of-teaching-platforms_fig1_380144052. [Accesat 29 mai 2025].
- [11] Twin Science, “5 Key Advantages of Using Teacher Platforms,” 2023. <https://www.twinscience.com/en/blog/5-key-advantages-of-using-teacher-platforms/>. [Accesat 2 iunie 2025].
- [12] Go Project, “Why Go?”. <https://go.dev/solutions/use-cases>. [Accesat 11 iunie 2025].
- [13] M. Farcas, “Why Go Is Popular Right Now and How and Why I Started Learning Go as a Node.js Developer,” 2024. dev.to/mihailtd/go-popular-now. [Accesat: 13 iunie 2025].
- [14] Startechup, “5 Benefits of Kubernetes for Business”, 2023. <https://www.startechup.com/blog/5-benefits-of-kubernetes/>. [Accesat: 14 iunie 2025].
- [15] J. John, “Why MongoDB? Exploring the Benefits and Use Cases of a Leading No-SQL Database,” 2024. dev.to/jottyjohn/why-mongodb. [Accesat: 16 iunie 2025].
- [16] SigNoz, “What Is the Advantage of Prometheus? Top 5 Advantages in 2025”, 2025. <https://signoz.io/guides/what-is-the-advantage-of-prometheus/>. [Accesat: 17 iunie 2025].
- [17] GetInRhythm, “Harnessing Grafana for Comprehensive Full-Stack Observability,” 2023. <https://medium.com/@GetInRhythm/harnessing-grafana-for-comprehensive-full-stack-observability-c63a37277044>. [Accesat: 18 iunie 2025].